

**Implementasi *Hybrid Intrusion Prevention System* Berbasis
Suricata dan Model *Extreme Gradient Boosting* (XGBoost)
Untuk Mitigasi Serangan SQL Injection**

SKRIPSI

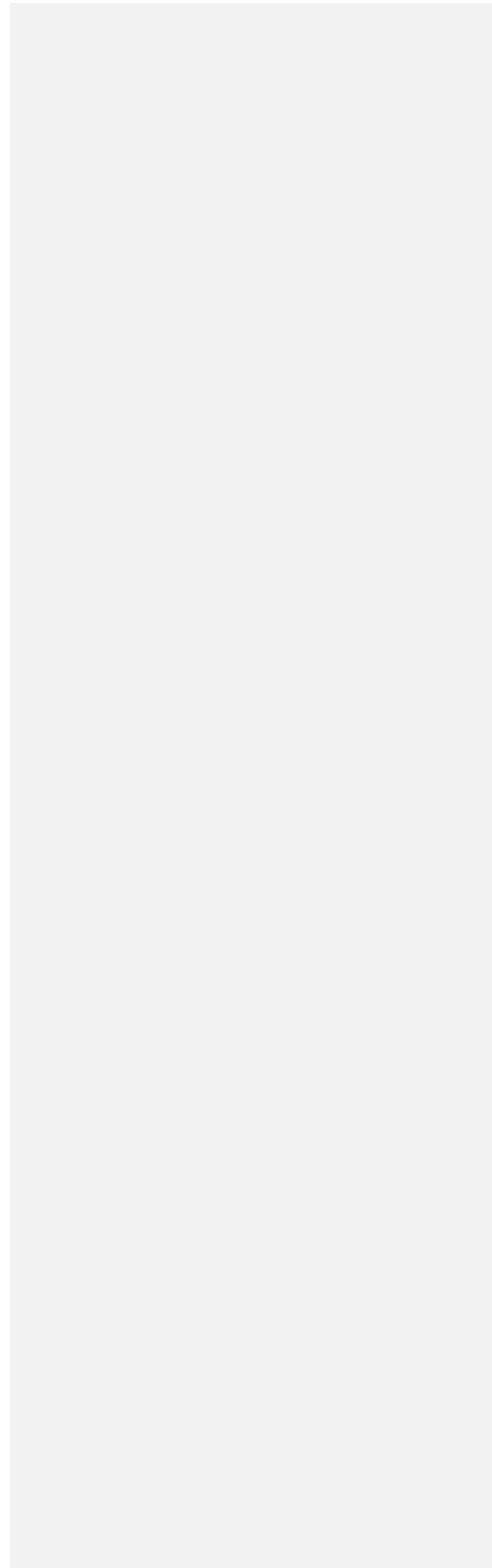
Disusun oleh:
Ahmad Nabih Baril Hilmy
NIM: 225150207111085



TEKNIK INFORMATIKA
TEKNIK INFORMATIKA
FAKULTAS ILMU KOMPUTER
UNIVERSITAS BRAWIJAYA
MALANG
2025

PENGESAHAN

Tanda tangan pengesahan



PERNYATAAN ORISINALITAS

Saya menyatakan dengan sebenar-benarnya bahwa sepanjang pengetahuan saya, di dalam naskah skripsi ini tidak terdapat karya ilmiah yang pernah diajukan oleh orang lain untuk memperoleh gelar akademik di suatu perguruan tinggi, dan tidak terdapat karya atau pendapat yang pernah ditulis atau diterbitkan oleh orang lain, kecuali yang secara tertulis disitasi dalam naskah ini dan disebutkan dalam daftar referensi.

Apabila ternyata di dalam naskah skripsi ini dapat dibuktikan terdapat unsur-unsur plagiasi, saya bersedia skripsi ini digugurkan dan gelar akademik yang telah saya peroleh (sarjana) dibatalkan, serta diproses sesuai dengan peraturan perundang-undangan yang berlaku (UU No. 20 Tahun 2003, Pasal 25 ayat 2 dan Pasal 70).

Malang,

Ahmad Nabih Baril Hilmy

NIM: 225150207111085

PRAKATA

Terima kasih kepada ...

ABSTRAK

ABSTRACT

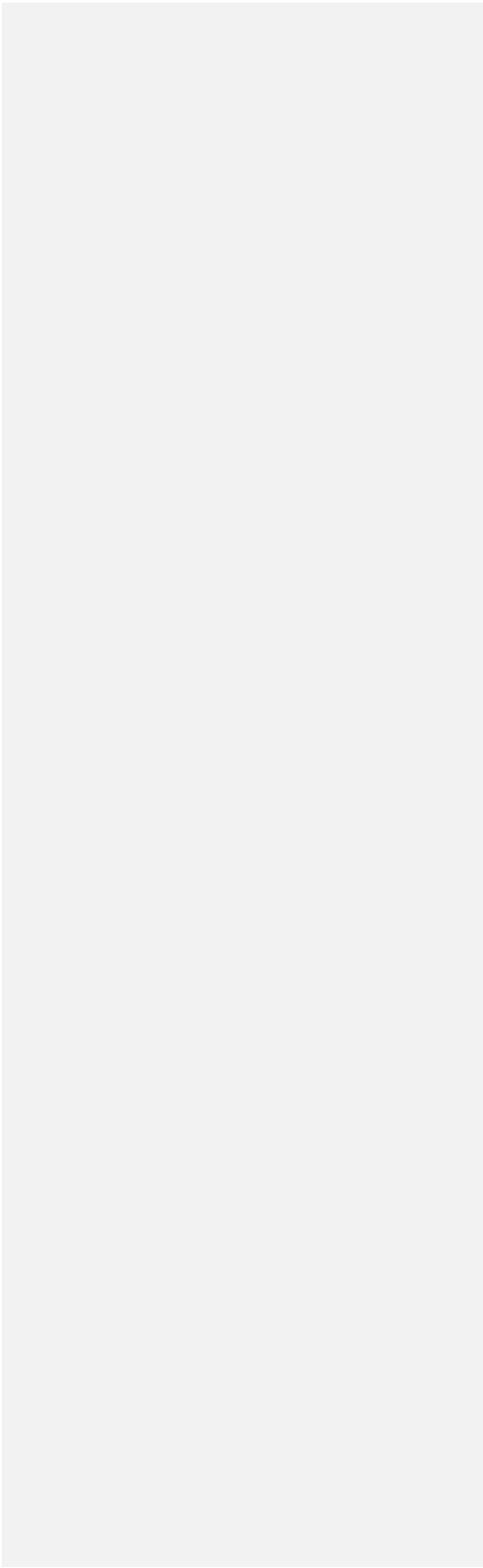
DAFTAR ISI

PENGESAHAN	2
PERNYATAAN ORISINALITAS	3
PRAKATA.....	4
ABSTRAK	5
ABSTRACT	6
DAFTAR ISI	7
DAFTAR TABEL.....	9
DAFTAR GAMBAR.....	10
BAB 1 PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	2
1.3 Manfaat	3
1.4 Tujuan.....	3
1.5 Batasan Masalah	4
1.6 Sistematika Pembahasan.....	5
BAB 2 LANDASAN KEPUSTAKAAN	7
2.1 Kajian Pustaka.....	7
2.2 Dasar Teori.....	12
2.2.1 Layanan Aplikasi Web	12
2.2.2 SQL Injection	13
2.2.3 Suricata.....	14
2.2.4 TF-IDF	17
2.2.5 Extreme Gradient Boosting (XGBoost).....	19
2.2.6 <i>Nfqueue</i>	20
2.2.7 Mesin Virtual.....	21
2.2.8 Parameter Evaluasi.....	22
BAB 3 METODOLOGI	25
3.1 Metode Penelitian	25
3.2 Studi Literatur	26
3.3 Perancangan Sistem	27

3.3.1	Perancangan Lingkungan Penelitian	28
3.3.2	Pengumpulan Data.....	29
3.3.3	Perancangan Aturan Suricata.....	30
3.3.4	Pelatihan Model Pembelajaran Mesin.....	38
3.4	Skenario Pengujian dan Evaluasi	39
3.4.1	Skenario Pengujian (BANYAK SKENARIO YANG BELUM DICERITAIN! TAPI JANGAN SAMPE KELUAR DARI LINGKUP PENELITIAN PLIS!)	39
3.4.2	Parameter Evaluasi.....	41
3.5	Pengambilan Kesimpulan	41
BAB 4	IMPLEMENTASI	42
4.1	Implementasi Lingkungan Penelitian	42
4.2	Pengumpulan Data	42
4.2.1	Pengumpulan Data Serangan SQL Injection	42
4.2.2	Pengumpulan Data Lalu Lintas Normal.....	42
4.3	Implementasi Pembuatan Aturan Suricata	42
4.4	Implementasi Model	43
4.4.1	Pelatihan Model TF-IDF.....	43
4.4.2	Pelatihan Model XGBoost	43
4.5	Pengujian dan Evaluasi	43
BAB 5	HASIL DAN ANALISIS	44
5.1	Hasil	44
5.2	Analisis.....	44
BAB 6	PENUTUP	46
6.1	Kesimpulan	46
6.2	Saran.....	46
DAFTAR PUSTAKA.....		47
LAMPIRAN		50

DAFTAR TABEL

DAFTAR GAMBAR



BAB 1 PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi yang pesat telah meningkatkan ketergantungan organisasi terhadap aplikasi web dan sistem basis data. Namun, peningkatan ini juga diikuti oleh eskalasi ancaman keamanan siber, khususnya serangan SQL Injection yang menempati posisi 3 teratas dalam OWASP Top 10 sebagai jenis kerentanan paling kritis (OWASP Foundation, 2023). SQL Injection merupakan teknik penyerangan yang memanfaatkan kerentanan *input validation* pada aplikasi web untuk memanipulasi *query* database, sehingga penyerang dapat mengakses, memodifikasi, atau menghapus data sensitif secara tidak sah (Alwan dan Younis, 2017). Dampak kerugian dari serangan SQL Injection sangat signifikan bagi organisasi. Statistik keamanan siber menunjukkan peningkatan drastis serangan berbasis web application, dengan SQL Injection menjadi salah satu vektor serangan utama ("160 Cybersecurity Statistics", 2022). Insiden nyata seperti kerentanan pada WooCommerce yang mengekspos 5 juta situs web (Ilascu, 2021), serangan pada aplikasi BillQuick untuk *deploy ransomware* (Cimpanu, 2021), dan kerentanan pada *framework* Django menunjukkan urgensi penanganan SQL Injection secara komprehensif (Sharma, 2022).

Sistem keamanan tradisional seperti *Intrusion Prevention System* (IPS) yang berbasis *signature detection*, termasuk Suricata, memiliki keterbatasan fundamental dalam menghadapi ancaman keamanan modern (Agoramoorthy dkk., 2023). Pendekatan *signature-based detection* mengandalkan *database* pola serangan yang telah diketahui sebelumnya, sehingga rentan terhadap teknik *bypass* yang menggunakan *encoding*, *obfuscation*, atau variasi sintaks yang tidak terdaftar dalam *signature database* (Aslan dan Samet, 2020). Lebih kritis lagi, sistem berbasis *signature* kesulitan dalam mendeteksi *zero-day attacks* atau varian serangan baru yang belum memiliki *signature*, menciptakan celah keamanan yang dapat dieksploitasi oleh penyerang (Guo, 2023). Keterbatasan lain dari sistem *signature-based* adalah kompleksitas dalam *maintenance* dan *update signature database* yang memerlukan intervensi manual secara berkala. Hal ini menyebabkan celah kerentanan dimana sistem tidak dapat mendeteksi serangan terbaru hingga *signature* yang relevan ditambahkan ke *database*. Selain itu, *signature-based detection* sering menghasilkan kesalahan deteksi dan peringatan yang cukup tinggi akibat pencocokan pola yang terlalu kaku, sehingga dapat mengganggu operasional normal aplikasi web.

Untuk mengatasi keterbatasan tersebut, diperlukan pendekatan yang lebih adaptif dan cerdas dalam mendeteksi serangan SQL Injection. *Machine Learning* menawarkan solusi promising dengan kemampuannya untuk belajar dari data dan mengidentifikasi pola serangan yang kompleks tanpa bergantung pada *signature* yang telah didefinisikan sebelumnya (Muhammad Arslan Ayub dkk., 2025). Di antara berbagai algoritma ML yang tersedia, XGBoost (*eXtreme Gradient Boosting*) telah terbukti memberikan performa superior dalam berbagai domain aplikasi,

Commented [AN1]: Tambahkan hipotesis terhadap penelitian sebelumnya terhadap apa yang akan kita kerjakan dalam penelitian kali ini (lihat Bab 3)

termasuk deteksi anomali dan klasifikasi serangan siber, XGBoost memiliki beberapa keunggulan dibandingkan algoritma ML lainnya, yaitu akurasi yang tinggi pada nilai 0.95 dalam proses *training* dan *inference* scalability yang baik untuk menangani *dataset* besar (De Carvalho Bertoli dkk., 2021). Algoritma ini menggunakan teknik gradient boosting yang mengoptimalkan gabungan dari *decision trees*, sehingga mampu menangkap pola kompleks dalam data dengan efisien. Diperkuat dengan dukungan pemrosesan vektorisasi pada karakter *query* menggunakan model *Natural Language Processing* TF-IDF yang membantu model mencapai akurasi pengujian yang sangat tinggi sebesar 99,37% (Pansare dkk., 2025). Karakteristik ini sangat sesuai untuk aplikasi dengan *real time detection* dimana kecepatan respon dan akurasi deteksi menjadi faktor kritis. Konsep sistem *hybrid* yang menggabungkan *signature-based detection* dengan *machine learning-based detection* menawarkan solusi komprehensif untuk mengatasi keterbatasan masing-masing pendekatan. Sistem *hybrid* dapat memanfaatkan kecepatan *signature matching* untuk serangan yang sudah dikenal, sambil menggunakan ML untuk mendeteksi serangan yang telah dimodifikasi dan lolos dari lapisan penanganan pada Suricata. Dengan integrasi XGBoost ke dalam arsitektur IPS diharapkan dapat menangkap *request* yang mengandung serangan SQL Injection yang dikirimkan oleh penyerang untuk meningkatkan kapabilitas deteksi sambil mempertahankan performa mendekati *real time* yang diperlukan untuk proteksi aplikasi web.

Hipotesis pada penelitian ini dibangun di atas titik temu antara kapabilitas reduksi *false positive* melalui arsitektur validasi berlapis dan efisiensi pendelegasian paket dari Suricata ke mesin klasifikasi lanjutan. Integrasi Suricata sebagai pencegahan pada *signature* awal dengan model XGBoost berbasis TF-IDF melalui antarmuka NFQUEUE diproyeksikan mampu mengidentifikasi variasi serangan *SQL Injection* khususnya pada muatan yang mengalami *multivariety encoding* dan *heavy tampering* secara lebih sensitif dibandingkan sistem statis murni. Pendelegasian paket HTTP yang lolos dari inspeksi statis ke dalam ruang vektor TF-IDF (Pansare dkk., 2025) diasumsikan akan mengeliminasi *blind spot* pendeteksian celah keamanan baru tanpa melampaui ambang batas toleransi respon pada sistem yang diharapkan mendekati waktu nyata.

1.2 Rumusan Masalah

Rumusan masalah pada penelitian ini adalah sebagai berikut.

1. Bagaimana merancang arsitektur sistem *hybrid* Intrusion Prevention System (IPS) yang mengintegrasikan metode *signature-based detection* pada Suricata dan *machine learning-based detection* pada XGBoost untuk mendeteksi serangan SQL Injection secara optimal?
2. Seberapa efektif sistem *hybrid* IPS yang memanfaatkan Suricata dan XGBoost dalam meningkatkan tingkat deteksi dan menurunkan angka *false positive* dibandingkan dengan IPS tradisional yang hanya menggunakan *signature-based detection*?

3. Bagaimana kinerja sistem dalam mengklasifikasikan *traffic normal* dan serangan SQL Injection, dari sisi akurasi, presisi, sensitivitas, dan F1-score?

1.3 Manfaat

Penelitian ini diharapkan memberikan manfaat yang bermakna baik pada ranah akademis maupun praktis. Dari sisi akademis, penelitian ini memperkaya kajian ilmiah terkait penerapan pembelajaran mesin dalam sistem deteksi dan pencegahan intrusi, khususnya pada konteks serangan SQL Injection. Integrasi antara Suricata, teknik pemrosesan teks berbasis TF-IDF, dan algoritma XGBoost memberikan sudut pandang baru mengenai efektivitas pendekatan *hybrid* dalam meningkatkan akurasi deteksi. Temuan yang diperoleh dapat menjadi rujukan bagi peneliti lain yang ingin mengembangkan metode serupa atau mengeksplorasi lebih jauh pemanfaatan model *gradient boosting* untuk aplikasi keamanan jaringan, sehingga turut memperkuat kontribusi penelitian di bidang teknologi keamanan siber.

Dari sisi praktis, sistem *hybrid IPS* yang dikembangkan dapat menjadi solusi yang relevan bagi organisasi yang membutuhkan perlindungan lebih adaptif terhadap serangan SQL Injection. Kombinasi antara aturan-aturan Suricata dan model XGBoost berbasis TF-IDF memungkinkan sistem mengenali pola serangan yang telah diketahui maupun variasi baru yang sering memanfaatkan teknik *encoding* atau *obfuscation*. Implementasi sistem ini berpotensi menurunkan risiko keberhasilan serangan dan meminimalkan gangguan operasional akibat kegagalan dan kesalahan deteksi yang berlebihan. Selain itu, rancangan arsitektur yang dihasilkan dapat menjadi kerangka kerja yang mudah diadaptasi untuk jenis serangan aplikasi web lainnya dengan penyesuaian minimal, sehingga memberikan nilai praktis yang tinggi bagi penerapan di lingkungan produksi. Penelitian ini juga menyediakan panduan konseptual dan teknis bagi praktisi keamanan siber dalam merancang dan mengimplementasikan mekanisme sistem *hybrid* yang responsif dan efisien.

1.4 Tujuan

Penelitian ini bertujuan mengembangkan sebuah arsitektur *Hybrid Intrusion Prevention System (IPS)* yang mampu memberikan kemampuan deteksi yang lebih adaptif terhadap serangan SQL Injection. Sistem *hybrid* ini dirancang dengan mengintegrasikan mekanisme *signature-based detection* pada Suricata dengan model pembelajaran mesin XGBoost yang diperkuat melalui pemrosesan teks berbasis TF-IDF. Kombinasi tersebut diharapkan dapat mengatasi keterbatasan pendekatan *signature* yang sering gagal mendeteksi variasi serangan berbasis *obfuscation* maupun pola sintaks yang tidak tercakup dalam aturan Suricata.

Tujuan berikutnya adalah menilai efektivitas pendekatan *hybrid* tersebut melalui serangkaian pengujian komparatif terhadap sistem IPS berbasis *signature*

murni. Penilaian difokuskan pada kemampuan sistem dalam meningkatkan tingkat deteksi serta menurunkan tingkat false positive pada lalu lintas HTTP yang mengandung pola SQL Injection. Kinerja sistem dievaluasi menggunakan metrik akurasi, *precision*, *recall*, dan F1-score yang dihasilkan dari data uji yang berasal dari simulasi serangan secara otomatis maupun percobaan manual, sehingga diperoleh gambaran yang objektif mengenai reliabilitas sistem dalam membedakan permintaan normal dan serangan.

Secara keseluruhan, penelitian ini bertujuan menghasilkan solusi *hybrid* IPS yang tidak hanya mampu menangani pola serangan yang telah diketahui, dan juga ketahanan yang lebih baik terhadap variasi dan bentuk serangan baru yang telah diperkuat, yang kerap muncul pada aplikasi web modern.

1.5 Batasan Masalah

Penelitian ini memiliki beberapa batasan yang ditetapkan untuk memperjelas ruang lingkup kajian dalam penelitian. Batasan tersebut merupakan aspek-aspek yang secara teoritis dapat dilakukan, namun tidak menjadi fokus dalam penelitian ini. Batasan-batasan tersebut adalah sebagai berikut:

1. Penelitian ini difokuskan pada deteksi serangan injeksi SQL yang disisipkan melalui bagian *query string* atau parameter URL pada lalu lintas data HTTP.
2. Mekanisme deteksi pada lapisan pembelajaran mesin menggunakan model XGBoost yang dipadukan dengan teknik pemrosesan teks TF-IDF.
3. Set data yang digunakan untuk pelatihan dan pengujian model terbatas pada lalu lintas HTTP yang dihasilkan melalui simulasi serangan dalam lingkungan eksperimen serta set data publik CIC-IDS2017 yang digunakan sebagai sumber lalu lintas normal.
4. Konsep serangan baru dalam penelitian ini merujuk pada serangan injeksi SQL yang tidak umum dan tidak dilakukan secara masif. Serangan tersebut menggunakan pola yang belum dikenali oleh sistem pertahanan tradisional.
5. Sistem hanya melakukan inspeksi dan mitigasi terhadap permintaan yang menggunakan protokol HTTP tanpa memerlukan mekanisme inspeksi TLS atau pendekatan tambahan lainnya.
6. Ruang lingkup pengujian terbatas pada lingkungan terkontrol berbasis virtualisasi. Evaluasi tidak mencakup skenario operasional berskala nyata atau pengujian beban lalu lintas jaringan, seperti pengukuran *throughput* dan latensi pada lalu lintas berskala besar yang terdapat pada layanan komersial publik.
7. Fitur yang dianalisis oleh model difokuskan pada isi kueri dan muatan HTTP. Penelitian ini tidak melakukan analisis secara menyeluruh terhadap struktur paket jaringan pada lapisan protokol yang lebih rendah.
8. Penggunaan Wireguard ditetapkan sebagai batasan infrastruktur untuk mengamankan jalur komunikasi dan meminimalkan derau data yang berasal dari sumber eksternal yang tidak terkendali selama proses penelitian.

1.6 Sistematika Pembahasan

Sistematika pembahasan pada penelitian ini adalah sebagai berikut.

BAB 1 PENDAHULUAN

Bab ini berisi gambaran umum mengenai penelitian, meliputi latar belakang, rumusan masalah, tujuan, manfaat, batasan masalah, serta sistematika pembahasan. Bab ini memberikan dasar pemahaman mengenai urgensi penelitian serta arah pengembangan sistem *hybrid* IPS yang menjadi focus utama dalam penelitian ini.

BAB 2 LANDASAN KEPUSTAKAAN

Bab ini memuat kajian teori dan penelitian terdahulu yang relevan dengan topik penelitian. Pembahasan mencakup konsep SQL Injection, mekanisme kerja Suricata sebagai *signature*-based IPS, teknik pemrosesan teks TF-IDF, algoritma XGBoost, arsitektur *NFQUEUE*, serta pendekatan *hybrid* detection yang digunakan pada penelitian sebelumnya. Kajian pustaka membangun fondasi teoretis yang mendukung perumusan metode penelitian

BAB 3 METODOLOGI

Bab ini menjelaskan metode penelitian yang digunakan, termasuk perancangan lingkungan uji berbasis virtualisasi, skenario simulasi serangan, proses pembentukan *dataset*, teknik prapemrosesan menggunakan TF-IDF, pelatihan model XGBoost, serta integrasi model dengan Suricata melalui mekanisme *NFQUEUE*. Selain itu, bab ini menguraikan prosedur pengujian dan evaluasi yang menghasilkan metrik kinerja sistem.

BAB 4 IMPLEMENTASI

Bab ini menguraikan implementasi teknis dari sistem *hybrid* IPS yang dikembangkan. Pembahasan mencakup konfigurasi Suricata dan ruleset yang digunakan, proses pembentukan antrean *NFQUEUE*, integrasi model XGBoost dengan modul inspeksi paket, serta pemaparan struktur arsitektur sistem yang dijelaskan secara rinci berdasarkan topologi jaringan penelitian. Setiap komponen dijelaskan sesuai alur kerja aktual pada sistem serta menjadi penghubung antara teori pada Bab 2 dan implementasi sistem pada Bab 4.

BAB 5 HASIL DAN ANALISIS

Bab ini menyajikan hasil pengujian serta analisis performa sistem *hybrid* IPS. Analisis meliputi efektivitas deteksi, tingkat false positive, serta evaluasi metrik seperti akurasi, precision, recall, dan F1-score berdasarkan *dataset* uji dan skenario serangan yang dijalankan. Bab ini juga membandingkan kinerja pendekatan *hybrid* dengan sistem berbasis

Commented [2]: Implementasi

signature murni, sehingga diperoleh pemahaman yang komprehensif terhadap kontribusi penelitian.

BAB 6 PENUTUP

Bab ini memuat kesimpulan yang diperoleh dari hasil penelitian serta saran untuk pengembangan lebih lanjut. Bagian ini merangkum temuan utama terkait efektivitas pendekatan *hybrid* dan memberikan arahan potensial untuk peningkatan sistem di masa mendatang, seperti perluasan jenis serangan, penggunaan model NLP berbasis embedding, atau uji coba pada lingkungan produksi.

BAB 2 LANDASAN KEPUSTAKAAN

Landasan kepustakaan berisi uraian dan pembahasan tentang teori, konsep, model, metode, atau sistem dari pustaka ilmiah, yang berkaitan dengan tema, masalah, atau pertanyaan penelitian. Dalam landasan kepustakaan terdapat landasan teori dari berbagai sumber pustaka yang terkait dengan teori dan metode yang digunakan dalam penelitian. Pembahasan berfokus kepada penjelasan mengenai serangan SQL Injection dan bagaimana penelitian terdahulu menanganinya menggunakan aturan-aturan Suricata, model TF-IDF dan XGBoost, serta pendekatan *hybrid* dalam konteks penanggulangan serangan SQL Injection.

2.1 Kajian Pustaka

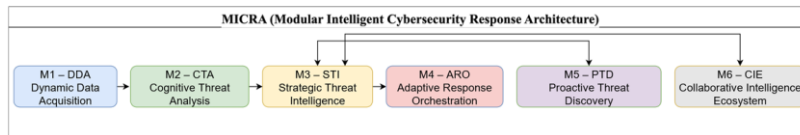
Terdapat beberapa penelitian terdahulu yang terkait dengan penanganan intrusi keamanan siber serta penerapan Machine Learning dalam konteks penanganan secara *hybrid*. Berikut beberapa penelitian yang dirujuk dan menjadi landasan dalam pelaksanaan penelitian pada sisi masalah, metode yang digunakan, hasil yang didapatkan dan relasi penelitian yang dilakukan terhadap penelitian ini.

No	Nama dan Tahun	Masalah	Metode	Hasil	Relasi
1	(Coutinho & Araújo, 2025)	Kemampuan solusi keamanan siber konvensional masih terbatas dalam menghadapi ancaman yang terus berevolusi. Sistem yang ada umumnya bersifat terkotak, sulit beradaptasi dengan pola serangan baru, memiliki tingkat <i>false positive</i> yang	Penelitian mengusulkan MICRA, yaitu arsitektur modular yang mengintegrasikan enam komponen utama: (1) Dynamic Data Acquisition untuk pengumpulan data jaringan dan honeypot; (2) Cognitive Threat Analysis yang memanfaatkan pembelajaran mesin, heuristik, dan analisis	MICRA menghasilkan model konseptual yang mampu mengkoordinasikan deteksi, validasi, serta respons ancaman secara adaptif dan terotomasi. Arsitektur ini menekankan fleksibilitas implementasi, peningkatan akurasi melalui multi-layer validation, deteksi ancaman laten melalui analisis	Penelitian ini memiliki relevansi pada aspek integrasi deteksi berbasis pembelajaran mesin dengan mekanisme pencegahan intrusi berbasis <i>signature</i> atau disebut dengan pendekatan <i>hybrid</i> . Selain itu, temuan mengenai pentingnya validasi berlapis, penurunan <i>false positive</i> , serta orkestrasi respons otomatis

		tinggi, serta kurang efektif dalam integrasi otomatis antara deteksi, validasi, respons, dan kolaborasi intelijen antar organisasi.	perilaku; (3) Strategic Threat Intelligence dengan validasi berlapis untuk menekan <i>false positive</i> ; (4) Adaptive Response Orchestration untuk mitigasi otomatis; (5) Proactive Threat Discovery berbasis analisis data historis; serta (6) Collaborative Intelligence Ecosystem untuk pertukaran data serangan terverifikasi.	retrospektif, serta peningkatan <i>cyber resilience</i> melalui skema berbagi indikator ancaman secara aman. Pada tahap prototipe, pendekatan ini menunjukkan efisiensi respons yang lebih baik terhadap ancaman dalam lingkungan jaringan dinamis.	memperkuat landasan teoritis dalam pengembangan sistem pencegahan serangan injeksi SQL yang adaptif pada arsitektur jaringan modern.
2	(Syamsuddin & Barukab, 2022)	Pertumbuhan perangkat IoT meningkatkan kompleksitas jaringan dan ancaman botnet yang semakin sulit dideteksi oleh IDS tradisional. Metode <i>signature</i> tidak mampu mengenali varian baru, sedangkan <i>anomaly-</i>	Penelitian menerapkan kombinasi Suricata IDS dengan algoritma KNN yang ditingkatkan menggunakan teknik seleksi fitur: <i>Information Gain</i> , <i>Forward Selection</i> , dan <i>Backward Elimination</i> pada <i>dataset</i> Bot IoT. Proses pemilihan	Teknik Forward Selection (KNN FS) menghasilkan performa terbaik: akurasi 99,89%, recall 97,82%, presisi 99,77%, F1-score 98,78%, dan waktu eksekusi tercepat 7 detik berdasarkan perbandingan pada RapidMiner.	Relevansi penelitian ini terletak pada penggunaan Suricata sebagai mesin deteksi berbasis <i>signature</i> yang dipadukan dengan model pembelajaran mesin untuk meningkatkan akurasi deteksi intrusi. Penekanan pada reduksi <i>false positive</i> serta

		<i>based</i> cenderung menghasilkan <i>false positive</i> tinggi. Selain itu, algoritma KNN memiliki kinerja rendah pada <i>dataset</i> berskala besar seperti Bot-IoT, sehingga diperlukan peningkatan performa klasifikasi khususnya pada lingkungan perangkat terbatas daya komputasi.	fitur, pelatihan model, evaluasi lima metrik (akurasi, presisi, recall, F1-score, waktu pemrosesan), serta implementasi model terbaik pada Raspberry Pi sebagai sistem IDS bernama SUKRY.	Hasil ini menunjukkan seleksi fitur efektif dalam menekan kompleksitas komputasi serta meningkatkan kemampuan klasifikasi botnet di lingkungan IoT nyata. Implementasi SUKRY berhasil menunjukkan bahwa IDS berbasis pembelajaran mesin dapat berjalan pada perangkat sumber daya terbatas (Raspberry Pi).	peningkatan efisiensi melalui seleksi fitur memberikan landasan penting dalam merancang sistem pencegahan serangan SQL Injection yang adaptif dan mampu beroperasi secara <i>inline</i> pada perangkat jaringan.
--	--	---	---	--	--

Pada penelitian oleh Coutinho & Araújo (2025), disebutkan bahwa ancaman siber dalam jaringan modern terus berkembang dengan karakteristik yang semakin dinamis dan sulit diprediksi. Sistem deteksi dan pencegahan intrusi tradisional umumnya masih bertumpu pada pencocokan pola serangan yang sudah dikenal sehingga tidak mampu mengantisipasi serangan baru yang belum memiliki *signature* yang terdokumentasi. Selain itu, mekanisme pertahanan yang terpisah antara proses deteksi, analisis, dan respons sering kali menimbulkan keterlambatan dalam mitigasi karena keputusan berada pada beberapa komponen yang tidak saling terintegrasi. Kondisi tersebut menyebabkan tingginya tingkat kesalahan deteksi, meningkatnya beban kerja analis keamanan, serta rendahnya pemanfaatan intelijen ancaman yang sebenarnya dapat memperkaya basis pengetahuan pertahanan jaringan. Penelitian MICRA mengawali gagasannya dari permasalahan tersebut, yaitu kebutuhan akan sebuah sistem yang mampu menghubungkan seluruh tahapan mulai dari pengumpulan data hingga pengambilan keputusan mitigasi secara cepat dan terverifikasi.



Gambar 2.1 Arsitektur Modular Intelligent Cybersecurity Response Architecture (MICRA)

Sebagai solusi atas permasalahan tersebut, penelitian MICRA mengembangkan sebuah kerangka kerja pertahanan jaringan yang bersifat modular dan adaptif dengan memadukan pembelajaran mesin, intelijen ancaman, dan mekanisme respons otomatis. Arsitektur MICRA terdiri atas enam komponen utama yang saling berkoordinasi. Komponen pertama adalah Dynamic Data Acquisition yang bertugas memastikan data ancaman diperoleh dari berbagai sumber termasuk sensor jaringan dan honeypot. Komponen kedua adalah Cognitive Threat Analysis yang menerapkan pembelajaran mesin, heuristik, dan analisis pola perilaku untuk menghasilkan deteksi awal terhadap aktivitas mencurigakan. Komponen ketiga bernama Strategic Threat Intelligence yang menjalankan proses validasi berlapis berdasarkan basis pengetahuan internal maupun eksternal sehingga hanya ancaman yang benar-benar terverifikasi yang akan ditangani sebagai insiden berbahaya. Tahap berikutnya berada pada Adaptive Response Orchestration yang mengaktifkan tindakan mitigasi otomatis sesuai tingkat risiko serangan, seperti pemutusan koneksi atau pembatasan lalu lintas. Selain itu, arsitektur ini memiliki Proactive Threat Discovery untuk melakukan analisis historis terhadap data jaringan sehingga ancaman yang lolos dari deteksi awal tetap dapat ditemukan. Komponen terakhir adalah Collaborative Intelligence Ecosystem yang memungkinkan berbagi indikator ancaman dengan entitas lain secara aman untuk meningkatkan ketahanan kolektif terhadap serangan.

Implementasi konsep MICRA menunjukkan hasil yang menjanjikan dalam meningkatkan efektivitas pertahanan jaringan. Proses validasi berlapis yang diterapkan membantu mengurangi tingkat kesalahan deteksi sehingga sistem tidak tergesa-gesa melakukan pemblokiran terhadap aktivitas yang sebenarnya sah. Efisiensi alur kerja yang terbangun dari integrasi sistem deteksi, analisis, dan respons juga mempercepat pengambilan keputusan mitigasi sehingga ruang serangan yang dimanfaatkan penyerang dapat dipersempit. Analisis retrospektif pada data memberikan kemampuan tambahan dalam menghadapi ancaman yang sebelumnya tidak dapat diklasifikasikan pada tahap deteksi awal. Selain itu, rancangan modular memungkinkan sistem tetap dapat dioperasikan pada lingkungan dengan karakteristik berbeda, mulai dari jaringan berskala kecil hingga infrastruktur pusat data yang lebih kompleks, tanpa harus melakukan perubahan keseluruhan sistem. Temuan ini memberi landasan bahwa arsitektur pertahanan yang cerdas dan adaptif dapat mengarah pada pengembangan sistem yang mampu mempertahankan dirinya sendiri dalam menghadapi ancaman yang terus berubah.

Sehingga dapat diketahui dan diterapkan bahwa hubungan antara penelitian MICRA dan topik penelitian ini terletak pada kesamaan paradigma dalam membangun sistem pencegahan intrusi yang tidak hanya mengandalkan satu jenis teknik deteksi. Penelitian ini mengombinasikan Suricata sebagai detektor berbasis *signature* dan model XGBoost sebagai detektor berbasis pembelajaran mesin yang keduanya saling melengkapi dalam mengenali pola serangan injeksi SQL pada trafik HTTP. Konsep validasi berlapis yang ditekankan dalam MICRA juga memberikan dasar teoritis untuk memastikan bahwa keputusan pemblokiran paket hanya dilakukan terhadap trafik yang benar-benar berindikasi serangan sehingga risiko pemblokiran trafik yang sah dapat diminimalkan. Selain itu, gagasan orkestrasi respons adaptif yang dimiliki MICRA sangat selaras dengan penggunaan *NFQUEUE* dalam sistem ini yang memungkinkan keputusan terotomasi terhadap paket jaringan. Prinsip-prinsip teknis tersebut menjadikan MICRA sebagai rujukan arsitektural yang relevan dalam merancang sistem intrusi pencegahan modern yang akurat sekaligus responsif terhadap serangan injeksi SQL.

Selanjutnya, berdasarkan penelitian yang dilakukan oleh Syamsuddin & Barukab (2022), dipaparkan masalah berupa pertumbuhan ekosistem Internet of Things mengubah jaringan menjadi sistem yang semakin padat, heterogen, dan sulit diamankan. Perangkat IoT yang memiliki kemampuan komputasi terbatas menjadikannya sasaran botnet modern yang kini lebih adaptif, bervariasi, dan sulit dideteksi hanya dengan teknik *signature*-based intrusion detection. Sistem IDS berbasis anomali memang mampu mengenali pola serangan baru, tetapi sering menghasilkan false positive yang tinggi sehingga mengurangi keandalannya dalam lingkungan nyata yang penuh variasi trafik. Selain itu, sebagian besar penelitian IDS berbasis machine learning masih berhenti pada tahap simulasi tanpa adanya implementasi langsung pada perangkat berbiaya rendah seperti Raspberry Pi. Kondisi ini mendorong penelitian SUKRY untuk menjawab dua persoalan sekaligus, yaitu meningkatkan performa deteksi KNN pada *dataset* IoT berukuran besar dan mewujudkan IDS cerdas yang benar-benar dapat dioperasikan pada perangkat edge yang terbatas sumber dayanya.

Penelitian ini menerapkan kombinasi teknik seleksi fitur dengan algoritma k-Nearest Neighbor untuk meningkatkan performa klasifikasi botnet pada *dataset* Bot-IoT yang berjumlah sekitar 3 juta entri dengan 46 fitur trafik jaringan. Tiga metode seleksi fitur bawaan RapidMiner, yaitu Information Gain, Forward Selection, dan Backward Elimination, diterapkan untuk mengurangi dimensi fitur sekaligus menekan beban komputasi. Hasil seleksi fitur menghasilkan tiga model pembandingan, yaitu KNN-IG, KNN-FS, dan KNN-BE. Seluruh model diuji berdasarkan lima indikator utama yang mencerminkan performa deteksi dalam IDS, yaitu akurasi, presisi, recall, nilai F1, dan waktu eksekusi. *Dataset* yang digunakan mewakili skenario IoT yang realistis dengan keberimbangan antara trafik normal dan anomali sehingga dapat menguji kemampuan generalisasi model secara lebih faktual dibandingkan studi yang hanya menggunakan *dataset* kecil atau tidak terstandarisasi.

Hasil evaluasi menunjukkan bahwa kombinasi KNN dan teknik Forward Selection adalah pendekatan terbaik. Model KNN-FS menghasilkan akurasi 99,89 persen, nilai presisi 99,77 persen, recall 97,82 persen, nilai F1 sebesar 98,78 persen, serta waktu eksekusi tercepat yaitu 7 detik pada *dataset* pengujian, seperti ditampilkan pada Tabel 6 halaman 14. Efisiensi komputasi ini terutama diperoleh melalui reduksi fitur yang signifikan (dari 46 menjadi 8 fitur), sehingga perhitungan jarak antar tetangga menjadi lebih ringan namun tetap relevan dalam memisahkan trafik botnet dan trafik IoT yang sah. Dibandingkan pendekatan tanpa seleksi fitur atau dengan eliminasi bertahap (Backward Elimination), KNN-FS terbukti lebih unggul baik dari sisi keakuratan maupun kecepatan eksekusi, menjadikannya kandidat yang layak untuk diimplementasikan pada perangkat kelas mikro seperti Raspberry Pi.

Penelitian SUKRY berkaitan erat dengan sistem yang dibangun pada penelitian ini. Keduanya sama-sama mengintegrasikan Suricata dengan model machine learning sebagai lapisan kecerdasan tambahan untuk mengatasi keterbatasan mekanisme *signature*-based. Konsep reduksi fitur dan optimasi KNN pada SUKRY menjadi landasan bahwa model machine learning pada perangkat edge harus mempertimbangkan aspek performa komputasi, khususnya ketika menangani aliran trafik besar seperti pada IDS. Selain itu, implementasi nyata pada Raspberry Pi memperkuat relevansi pendekatan *hybrid* IDS dalam lingkungan produksi, sejalan dengan penelitian ini yang mengintegrasikan Suricata, *NFQUEUE*, dan model pembelajaran mesin sebagai mekanisme keputusan otomatis terhadap paket yang berindikasi serangan SQL Injection. Dengan demikian, SUKRY memberikan referensi empiris maupun arsitektural dalam merancang IPS *hybrid* yang akurat, efisien, dan siap diterapkan di jaringan nyata.

2.2 Dasar Teori

Bagian dasar teori ini disusun untuk memberikan landasan konseptual dan teknis yang mendukung perancangan serta implementasi sistem yang dikembangkan dalam penelitian ini. Pembahasan dimulai dari konsep layanan aplikasi website sebagai objek yang dilindungi, diikuti dengan karakteristik dan mekanisme serangan SQL Injection sebagai ancaman utama yang ditangani. Selanjutnya, dijelaskan peran Suricata sebagai sistem deteksi dan pencegahan intrusi berbasis *signature*, serta konsep ekstraksi fitur menggunakan TF-IDF dan proses klasifikasi menggunakan XGBoost sebagai komponen *machine learning*. Integrasi sistem dilakukan melalui mekanisme *Nfqueue* untuk memungkinkan inspeksi dan pengambilan keputusan secara inline, sementara penggunaan mesin virtual mendukung implementasi dan pengujian sistem dalam lingkungan terisolasi. Evaluasi kinerja sistem dilakukan menggunakan confusion matrix untuk mengukur efektivitas deteksi dan mitigasi serangan yang dihasilkan.

2.2.1 Layanan Aplikasi Web

Layanan aplikasi web merupakan komponen yang menyediakan fungsi tertentu kepada pengguna melalui mekanisme permintaan dan respons berbasis

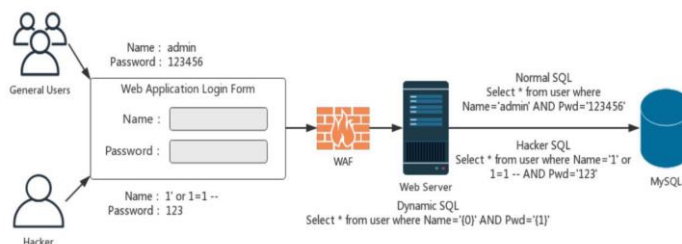
Commented [AN3]: Cerita deskriptif alur sistem pada dasar teori jelasin disini

protokol HTTP (Lirzin & Markhoff, t.t.). Pada konteks layanan modern modern, setiap permintaan dari pengguna, seperti pengisian formulir, pencarian produk, atau proses login, diteruskan ke server untuk diproses oleh aplikasi sebelum menghasilkan respon yang sesuai. Struktur dasar aplikasi website umumnya terdiri atas tiga lapisan utama yaitu, *presentation layer* yang mengelola tampilan dan antarmuka pengguna, *application logic* yang menangani proses bisnis, serta *database layer* yang menyimpan dan mengolah data (Ding, 2020). Keterhubungan antar lapisan ini menciptakan jalur komunikasi yang harus dijaga keamanannya.

Karena layanan aplikasi *website* sangat bergantung pada integritas komunikasi antara klien dan server, mekanisme pemantauan dan pencegahan intrusi menjadi kebutuhan yang tidak dapat diabaikan. Sistem keamanan, baik yang berbasis aturan maupun *machine learning*, berperan dalam menganalisis pola permintaan yang masuk untuk membedakan lalu lintas normal dari aktivitas yang bersifat mencurigakan. Di sinilah pendekatan *hybrid* yang dikembangkan pada penelitian ini menjadi penting. Pemahaman mengenai karakteristik permintaan HTTP, struktur parameter, serta bagaimana layanan web memproses input menjadi fondasi utama untuk mendeteksi anomali yang mengarah pada serangan SQL Injection.

Dengan demikian, pembahasan mengenai layanan aplikasi website menjadi dasar yang menghubungkan alur penelitian mulai dari pemahaman sumber kerentanan, mekanisme serangan SQL Injection, hingga kebutuhan akan sistem deteksi dan pencegahan berbasis Suricata, *NFQUEUE*, dan model XGBoost. Pemahaman ini memastikan setiap komponen pada sistem *hybrid* dapat disusun secara konsisten dan selaras dengan karakteristik lalu lintas aplikasi web yang menjadi objek penelitian.

2.2.2 SQL Injection



Gambar 2.2 Proses Serangan SQL Injection, oleh Chen dkk. (2021)

SQL Injection merupakan jenis serangan yang menargetkan aplikasi web dengan cara menyisipkan instruksi SQL berbahaya ke dalam parameter input yang diterima oleh server (Abdullayev & Chauhan, 2023). Serangan ini terjadi ketika mekanisme validasi atau sanitasi input tidak berjalan sebagaimana mestinya sehingga aplikasi salah menginterpretasikan data pengguna sebagai bagian dari

perintah *query*. Serangan SQL Injection dapat muncul dalam berbagai bentuk, seperti *union-based*, *boolean-based*, *error-based*, dan *time-based*, yang masing-masing memanfaatkan celah logika aplikasi untuk memperoleh informasi internal atau mengendalikan sistem basis data (Chen dkk., 2021). Variasi serangan ini terus berkembang seiring munculnya teknik *obfuscation* dan encoding, sehingga pola serangannya tidak selalu mudah dikenali melalui aturan statis.

Potensi bahaya SQL Injection terletak pada kemampuannya untuk mengubah struktur *query* asli dan memberikan akses langsung kepada penyerang terhadap data sensitif (Kareem dkk., 2021a). Pada layanan aplikasi website, parameter sederhana seperti *username*, *password*, atau *id* dapat dimanipulasi sehingga menghasilkan perintah yang tidak diantisipasi oleh aplikasi. Misalnya, masukan "admin' OR '1'='1" yang berpotensi melewati otentikasi, sementara pola *time-delay* seperti penggunaan *query* "SLEEP(5)" digunakan untuk menguji respons server (Kareem dkk., 2021). Ragam teknik ini menunjukkan bahwa SQL Injection bukan hanya satu bentuk serangan tunggal, tetapi merupakan keluarga serangan dengan variasi sintaks dan strategi eskalasi yang luas.

Selain potensi eksploitasi statis oleh penyerang manusia, SQL Injection juga memiliki alat pengujian otomatis yang dapat memetakan dan melakukan eksploitasi terhadap sistem dengan kerentanan SQL Injection, alat ini adalah Sqlmap (Q. Li dkk., 2024). Perangkat lunak sumber terbuka ini dirancang untuk mengotomatisasi proses deteksi dan eksploitasi celah keamanan *SQL Injection* pada sistem manajemen basis data. Sqlmap memiliki kemampuan untuk melakukan pemetaan basis data secara mendalam, mulai dari identifikasi sidik jari (*fingerprinting*) hingga ekstraksi data secara masif dari server target.

Karena serangan SQL Injection dapat ditulis dalam banyak bentuk dan dimodifikasi dengan berbagai teknik penyamaran, deteksi berbasis *signature* saja sering kali tidak cukup. Pola serangan dapat ditulis ulang menggunakan encoding URL, penggantian karakter, atau penyisipan komentar untuk menghindari pencocokan aturan statis. Oleh karena itu, penelitian ini mengembangkan pendekatan *hybrid* yang memadukan deteksi *signature* dengan analisis berbasis *machine learning*. Pemahaman mengenai sifat dinamis dan variatif SQL Injection menjadi fondasi dalam merancang sistem Suricata sebagai pertahanan tahap pertama dan mekanisme *machine learning* yang bertugas menangkap pola serangan yang tidak dapat ditangani oleh *signature* tradisional.

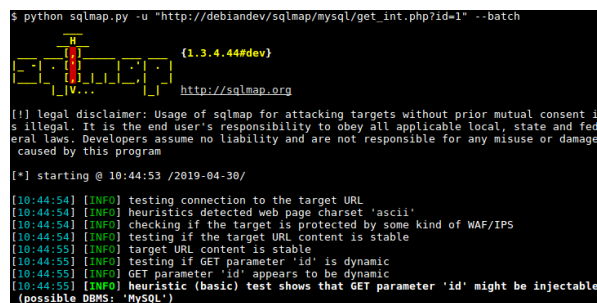
2.2.3 Alat Pengujian Permintaan query SQL

Dalam proses pengembangan dan pengujian sistem yang dikembangkan dalam penelitian ini, diperlukan alat yang mampu mengirimkan berbagai bentuk permintaan query SQL ke aplikasi target. Permintaan tersebut mencakup baik query yang bersifat normal maupun query yang mengandung pola serangan SQL Injection. Penggunaan alat pengujian ini bertujuan untuk mensimulasikan kondisi interaksi nyata antara pengguna atau penyerang dengan aplikasi web, sehingga mekanisme deteksi dan pencegahan yang diterapkan pada sistem dapat dievaluasi secara menyeluruh. Oleh karena itu, penelitian ini memanfaatkan beberapa alat

pengujian yang umum digunakan dalam pengujian keamanan aplikasi web. Adapun alat yang digunakan dalam penelitian ini adalah sebagai berikut.

1. Sqlmap

Merupakan perangkat lunak *open-source* yang dirancang untuk melakukan pengujian keamanan aplikasi web dengan fokus pada identifikasi dan eksploitasi kerentanan SQL Injection secara otomatis (B. Kalaiselvi dkk., 2025). Alat ini bekerja dengan mengirimkan berbagai variasi payload SQL ke parameter yang terdapat pada permintaan HTTP, kemudian menganalisis respons dari server untuk menentukan apakah suatu aplikasi rentan terhadap serangan SQL Injection.



```
python sqlmap.py -u "http://debiandev/sqlmap/mysql/get_int.php?id=1" --batch
[1.3.4.44#dev]
http://sqlmap.org

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is
illegal. It is the end user's responsibility to obey all applicable local, state and fed
eral laws. Developers assume no liability and are not responsible for any misuse or damage
caused by this program

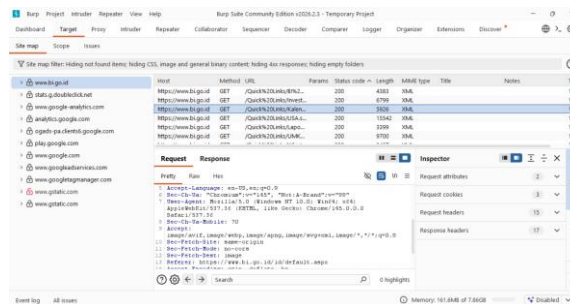
[*] starting @ 10:44:53 /2019-04-30/
[10:44:54] [INFO] testing connection to the target URL
[10:44:54] [INFO] heuristics detected web page charset 'ascii'
[10:44:54] [INFO] checking if the target is protected by some kind of WAF/IPS
[10:44:54] [INFO] testing if the target URL content is stable
[10:44:55] [INFO] target URL content is stable
[10:44:55] [INFO] testing if GET parameter 'id' is dynamic
[10:44:55] [INFO] GET parameter 'id' appears to be dynamic
[10:44:55] [INFO] heuristic (basic) test shows that GET parameter 'id' might be injectable
(possible DBMS: 'MySQL')
```

Selain kemampuan deteksi, Sqlmap juga menyediakan fitur otomatisasi dalam proses pengujian keamanan, seperti enumerasi struktur basis data, identifikasi jenis *database management system* yang digunakan, hingga pengambilan data dari tabel tertentu apabila kerentanan berhasil ditemukan (B. Kalaiselvi dkk., 2025). Dalam konteks penelitian ini, Sqlmap dimanfaatkan sebagai alat untuk menghasilkan lalu lintas permintaan yang mengandung pola serangan SQL Injection secara terstruktur dan sistematis, sehingga dapat digunakan untuk menguji kemampuan sistem dalam mendeteksi serta menangani permintaan berbahaya yang dikirimkan ke aplikasi web.

2. Burpsuite

Burp Suite merupakan platform pengujian keamanan aplikasi web yang banyak digunakan dalam proses analisis dan pengujian kerentanan pada sistem berbasis web. Perangkat lunak ini menyediakan berbagai modul yang memungkinkan peneliti atau penguji keamanan untuk memantau, memodifikasi, serta mengirimkan ulang permintaan HTTP dan HTTPS antara klien dan server. Salah satu komponen utama dari Burp Suite adalah intercepting proxy, yang memungkinkan pengguna untuk

menangkap dan mengubah isi permintaan sebelum diteruskan ke server tujuan (Kondraciuk dkk., 2022).



Burpsuite memiliki fitur-fitur seperti Proxy, Repeater, dan Intruder dimana fitur ini memungkinkan pengujian dilakukan secara lebih terkontrol dengan cara memodifikasi parameter pada permintaan HTTP, termasuk penyisipan payload SQL Injection pada bagian tertentu dari query (Qadir dkk., 2025). Dengan demikian, alat ini dapat digunakan untuk mensimulasikan berbagai bentuk permintaan yang mungkin dikirimkan oleh pengguna maupun penyerang. Dalam penelitian ini, Burp Suite digunakan untuk menghasilkan serta memodifikasi permintaan HTTP secara manual maupun semi-otomatis, sehingga variasi query yang akan diproses oleh sistem dan dilanjutkan untuk melalui lapisan IPS berikutnya pada Suricata.

2.2.4 Suricata

Suricata merupakan sistem keamanan jaringan berbasis *signature* yang berfungsi sebagai garis pertahanan pertama dalam arsitektur *hybrid* Intrusion Prevention System pada penelitian ini. Suricata bekerja dengan memeriksa setiap paket jaringan dan mencocokkannya dengan aturan atau *ruleset* yang telah ditentukan. Jika pola permintaan HTTP menyerupai *signature* yang dikenal sebagai bagian dari serangan SQL Injection, Suricata dapat langsung mengambil tindakan, seperti menandai, memblokir, atau meneruskan paket tersebut ke antrean pemrosesan lebih lanjut (Asad dkk., 2024). Dengan kemampuan ini, Suricata berperan sebagai lapisan keamanan yang memblokir serangan yang sudah diketahui dan terdokumentasi dalam *signature*.

Dalam konteks penelitian ini, Suricata ditempatkan sebagai *firewall tahap pertama* yang melakukan inspeksi awal terhadap lalu lintas HTTP. Posisi ini penting karena mayoritas serangan yang telah umum dan memiliki pola tetap dapat dihentikan tanpa perlu melibatkan proses komputasi yang lebih berat pada lapisan machine learning. Namun, *signature*-based detection memiliki keterbatasan, terutama saat menghadapi varian serangan baru atau pola serangan yang disamarkan melalui teknik *obfuscation* (Pansare dkk., 2025). Kemudian pada penelitian ini, ketika Suricata mendeteksi anomali atau ketika payload tidak cocok

dengan *signature* apa pun tetapi tetap mencurigakan, paket tersebut dialihkan melalui mekanisme *queue* pada *Nfqueue* untuk diperiksa lebih lanjut oleh modul *machine learning*. Peran Suricata yang hanya dapat mengenali pola-pola eksplisit menjadi alasan mengapa arsitektur *hybrid* diperlukan. Kombinasi antara kemampuan pencocokan *signature* yang cepat dan analisis adaptif berbasis *machine learning* menciptakan sistem yang lebih responsif terhadap ragam bentuk serangan SQL Injection yang terus berevolusi (Pansare dkk., 2025). Oleh karena itu, keluaran dari Suricata bukan hanya keputusan deteksi, tetapi juga bagian dari alur kerja menuju tahap selanjutnya.

2.2.5 TF-IDF

Term Frequency–Inverse Document Frequency (TF-IDF) merupakan salah satu teknik representasi teks digunakan dalam pemodelan machine learning untuk mengubah data berbasis karakter atau kata menjadi bentuk numerik (Christian dkk., 2016). Dalam penelitian ini, TF-IDF berperan sebagai tahap ekstraksi fitur yang berada di tengah alur sistem. Prosesnya menghubungkan tahap prapemrosesan teks dengan tahap klasifikasi menggunakan algoritma XGBoost. Karena XGBoost tidak dapat memproses teks secara langsung, TF-IDF menjadi komponen penting agar model dapat membaca dan mengenali pola pada *query* SQL yang masuk.

TF-IDF memberikan bobot pada setiap istilah, atau yang disebut dengan *term*, dengan mempertimbangkan dua hal:

1. Seberapa sering istilah tersebut muncul dalam satu *query*, dan
2. Seberapa jarang istilah tersebut digunakan pada keseluruhan *dataset*.

Sehingga dihasilkan bobot tinggi pada istilah yang khas untuk serangan SQL Injection, misalnya "UNION SELECT, SLEEP(5)", atau "OR 1=1 --" karena istilah tersebut jarang muncul pada lalu lintas normal. Sebaliknya, istilah umum pada masukan pengguna mendapatkan bobot rendah karena tidak memiliki nilai diskriminatif yang kuat (Y. Li & Zhang, 2019).

Perhitungan TF-IDF dilakukan dalam dua tahap. Komponen pertama adalah *Term Frequency* (TF) yang menghitung frekuensi kemunculan istilah (*t*) dalam sebuah *query* (*d*), dapat dilihat dalam persamaan (x.x) berikut ini:

$$TF(t, d) = \frac{f_{t,d}}{\text{total istilah dalam } d}$$

Keterangan:

t: istilah (*term*) berupa kata, simbol, atau token dari *query* SQL.

d: dokumen atau satu *query* SQL yang sedang dihitung bobotnya.

$f_{t,d}$: jumlah kemunculan istilah (*t*) di dalam dokumen(*d*).

Pada formula perhitungan TF, nilai frekuensi kemunculan istilah (*t*) dihitung dengan membandingkan jumlah kemunculan istilah tersebut terhadap total seluruh istilah yang terdapat dalam satu *query* (*d*). Pendekatan ini digunakan

untuk menormalkan nilai frekuensi sehingga *query* yang memiliki panjang berbeda tetap dapat dibandingkan secara adil. Semakin sering suatu istilah muncul di dalam satu *query*, maka nilai TF untuk istilah tersebut akan semakin besar. Namun, dengan membaginya terhadap total istilah dalam *query*, sistem mencegah bias yang dapat muncul akibat perbedaan panjang *query*. Dalam konteks SQL Injection, istilah atau simbol tertentu yang muncul berulang kali, seperti operator logika atau kata kunci SQL, akan memiliki nilai TF yang lebih tinggi dan mencerminkan tingkat dominasi istilah tersebut dalam satu permintaan. Nilai TF ini kemudian digunakan sebagai dasar untuk menentukan seberapa besar kontribusi sebuah istilah dalam membentuk karakteristik *query* sebelum dikombinasikan dengan komponen IDF.

Komponen kedua adalah *Inverse Document Frequency* (IDF), yang mengukur apakah istilah tersebut umum atau sebenarnya cukup jarang digunakan pada seluruh *dataset*, dapat dilihat dalam persamaan (x.x) berikut ini:

$$IDF(t) = \log\left(\frac{N}{1 + df(t)}\right)$$

Keterangan:

N : jumlah seluruh dokumen atau *query* dalam *dataset*.

$df(t)$: jumlah dokumen yang mengandung istilah(t).

$\log$: logaritma (*base e*) digunakan untuk mengatur skala nilai IDF agar tidak terlalu besar.

Dimana dapat diketahui, pada IDF, nilai kelangkaan suatu istilah (t) ditentukan berdasarkan seberapa banyak dokumen atau *query* dalam *dataset* yang mengandung istilah tersebut. Semakin sedikit jumlah dokumen yang memuat istilah tertentu, maka nilai IDF yang dihasilkan akan semakin besar. Hal ini menunjukkan bahwa istilah tersebut bersifat spesifik dan tidak umum digunakan. Sebaliknya, istilah yang muncul pada hampir seluruh dokumen akan memiliki nilai IDF yang rendah karena dianggap kurang informatif. Penggunaan fungsi logaritma bertujuan untuk mengendalikan skala nilai IDF agar tidak tumbuh terlalu besar, sehingga perbedaan bobot antaristilah tetap proporsional. Dalam konteks deteksi SQL Injection, istilah atau simbol yang jarang muncul pada *query* normal tetapi sering muncul pada *query* serangan akan memperoleh nilai IDF yang tinggi, sehingga memiliki kontribusi lebih besar dalam proses klasifikasi.

Sehingga kemudian nilai akhir dihitung dengan mengalikan keduanya, sebagaimana persamaan (x.x):

$$TF - IDF(t, d) = TF(t, d) \times IDF(t)$$

Keterangan:

$TF(t, d)$: bobot frekuensi istilah dalam dokumen.

$IDF(t)$: jumlah dokumen yang mengandung istilah(t).

Nilai akhir TF-IDF diperoleh dengan mengalikan nilai Term Frequency dan Inverse Document Frequency. Perkalian ini bertujuan untuk menggabungkan

informasi lokal dan global dari sebuah istilah. TF merepresentasikan seberapa dominan istilah tersebut dalam satu *query*, sedangkan IDF merepresentasikan tingkat keunikan istilah tersebut pada keseluruhan *dataset*. Dengan mengombinasikan kedua nilai tersebut, TF-IDF mampu memberikan bobot tinggi pada istilah yang sering muncul dalam satu *query* tertentu namun jarang ditemukan pada *query* lain. Karakteristik ini sangat sesuai untuk mengidentifikasi pola SQL Injection, karena *query* serangan umumnya mengandung istilah atau simbol yang khas dan tidak lazim digunakan pada lalu lintas normal. Bobot TF-IDF inilah yang kemudian digunakan sebagai representasi numerik input bagi model XGBoost dalam menentukan apakah suatu permintaan tergolong normal atau berbahaya.

2.2.6 Extreme Gradient Boosting (XGBoost)

Extreme Gradient Boosting (XGBoost) termasuk kedalam algoritma *supervised learning* yang berbasis pada penggabungan banyak pohon keputusan (*ensemble decision trees*). Dalam penelitian ini, XGBoost berperan sebagai model keputusan utama yang mendukung Suricata. Jika Suricata tidak menjatuhkan paket melalui *signature*, XGBoost bertugas untuk memeriksa payload HTTP dan menentukan apakah paket tersebut merupakan serangan SQL Injection atau permintaan normal.

XGBoost tidak memproses teks HTTP secara langsung. Oleh karena itu, data teks terlebih dahulu melalui tahap prapemrosesan dan transformasi fitur. Teknik TF-IDF digunakan untuk mengubah teks menjadi vektor numerik berdasarkan tingkat kepentingan setiap istilah dalam suatu permintaan HTTP. Hasil transformasi tersebut membentuk vektor fitur x_i yang digunakan sebagai masukan model XGBoost. Setiap elemen dalam vektor menunjukkan bobot suatu istilah atau token yang terdapat pada permintaan HTTP. Dengan demikian, pola seperti kata kunci SQL, operator logika, karakter khusus, komentar SQL, dan kombinasi token yang dapat dipelajari oleh model berdasarkan representasi numeriknya.

Secara konsep, XGBoost membangun model secara bertahap. Setiap pohon keputusan baru dilatih untuk memperbaiki kesalahan prediksi dari pohon sebelumnya. Proses ini dikenal sebagai *gradient boosting*, yaitu pendekatan yang menggerakkan model menuju prediksi yang lebih akurat dengan meminimalkan kesalahan (*loss function*). Keunggulan utama XGBoost adalah efisiensi, kemampuan menangani *dataset* besar, serta mekanisme regularisasi yang menjaga model tetap stabil dan tidak mudah overfitting.

Secara matematis, prediksi akhir XGBoost diperoleh dengan menjumlahkan keluaran dari seluruh pohon keputusan yang dibangun, sebagaimana dirumuskan pada Persamaan (x.x) berikut:

$$\hat{y}_i = \sum_{k=1}^K f_k(x_i)$$

dengan $(\hat{y}_i)$ menyatakan hasil prediksi untuk sampel ke (i) , (K) sebagai jumlah total pohon keputusan, f_k sebagai fungsi pohon keputusan ke (k) , dan (x_i) sebagai vektor fitur hasil transformasi data. Persamaan ini menunjukkan bahwa keputusan XGBoost tidak ditentukan oleh satu pohon tunggal, melainkan oleh akumulasi kontribusi dari seluruh pohon yang telah dilatih.

Proses pelatihan pada algoritma XGBoost didasarkan pada prinsip *additive training*, di mana model mempelajari pola data secara iteratif dengan menambahkan pohon keputusan baru yang mampu memperbaiki kesalahan prediksi dari pohon sebelumnya. Tujuan utama dari proses ini adalah untuk meminimalkan fungsi kerugian. Secara matematis, fungsi objektif yang dioptimalkan dalam XGBoost dirumuskan sebagai persamaan (x.x) berikut:

$$\mathcal{L} = \sum_i l(y_i, \hat{y}_i) + \sum_k \Omega(f_k)$$

Komponen pertama $l(y_i, \hat{y}_i)$ yang merepresentasikan kesalahan antara nilai prediksi dan label sebenarnya, sedangkan $\Omega(f_k)$ merupakan fungsi regularisasi yang digunakan untuk mengendalikan kompleksitas model. Fungsi regularisasi pada XGBoost dirumuskan sebagai persamaan (x.x):

$$\Omega(f_k) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2$$

dengan T menyatakan jumlah daun pada pohon keputusan dan w_j sebagai bobot prediksi pada masing-masing daun. Parameter γ dan λ berfungsi untuk menekan kompleksitas model sehingga mengurangi risiko *overfitting*. Dengan formulasi tersebut, XGBoost mampu mempelajari pola serangan SQL Injection secara bertahap melalui kombinasi banyak pohon keputusan, sekaligus menjaga keseimbangan antara akurasi dan stabilitas model.

Dalam sistem yang dikembangkan, hasil klasifikasi XGBoost menjadi dasar bagi modul mitigasi untuk menentukan tindakan terhadap permintaan HTTP. Ketika probabilitas serangan memenuhi atau melampaui nilai ambang, permintaan dikategorikan sebagai serangan injeksi SQL dan diteruskan kepada mekanisme mitigasi. Sebaliknya, permintaan yang memiliki probabilitas di bawah nilai ambang dikategorikan sebagai lalu lintas normal. Integrasi tersebut memungkinkan sistem mendeteksi pola serangan yang tidak tercakup dalam aturan atau *signature* Suricata.

2.2.7 Nfqueue

Nfqueue adalah mekanisme antrean yang disediakan oleh Netfilter pada Linux untuk mengalihkan paket dari kernel ke sisi pengguna (*Netfilter*, 2022). Mekanisme ini memungkinkan aplikasi eksternal melakukan inspeksi tambahan sebelum kernel memutuskan apakah paket tersebut akan diterima, ditolak, atau

dimodifikasi. Dalam penelitian ini, *Nfqueue* berfungsi sebagai penghubung yang mengalirkan paket dari Suricata menuju modul machine learning sehingga kedua sistem keputusan dapat bekerja secara berurutan. Suricata, yang berada pada lapisan pertama, melakukan deteksi berbasis *signature*. Jika paket dianggap mencurigakan atau tidak cocok dengan aturan tertentu, paket tersebut diarahkan ke *Nfqueue*.

Dengan memanfaatkan pola tersebut, Suricata dapat dijalankan dalam mode *queue* atau *inline* sehingga posisinya berada langsung pada jalur lalu lintas (Open Information Security Foundation (OISF, 2025)). Dalam mode ini, paket-paket yang sesuai dengan aturan tertentu pada *ruleset* akan diteruskan ke antrean spesifik untuk diproses lebih lanjut. Tahapan selanjutnya dapat diintegrasikan dengan sistem yang mengimplementasikan model *machine learning*. Aplikasi ini melakukan pra pemrosesan terhadap *payload* atau *header* paket, kemudian menjalankan inferensi berdasarkan model yang telah dilatih. Hasil inferensi tersebut digunakan untuk memberikan keputusan akhir apakah paket dianggap normal dan diteruskan, atau dikategorikan sebagai ancaman dan diblokir.

2.2.8 Mesin Virtual

Kemudian, mesin virtual adalah sistem komputasi yang menjalankan sistem operasi dan aplikasinya di atas lapisan virtualisasi, bukan langsung pada perangkat keras fisik. Virtualisasi memungkinkan satu perangkat keras fisik menjalankan beberapa lingkungan komputasi yang berdiri sendiri, masing-masing dengan alokasi CPU, memori, penyimpanan, dan konfigurasi jaringan tersendiri. Pada tingkat teknis, mesin virtual bekerja melalui hypervisor yang bertugas membagi sumber daya fisik dan menghadirkannya seolah-olah setiap VM memiliki perangkat kerasnya sendiri. Pendekatan ini memberikan isolasi penuh antar VM, sehingga kesalahan atau serangan pada satu mesin tidak memengaruhi mesin lainnya.

Virtualisasi banyak digunakan dalam penelitian keamanan jaringan karena mempermudah pembuatan lingkungan uji yang kompleks tanpa memerlukan banyak perangkat fisik. Dengan VM, skenario jaringan dapat dibentuk secara lebih fleksibel, mulai dari konfigurasi topologi, simulasi serangan, hingga pengamatan lalu lintas. Selain itu, seluruh kondisi sistem dapat direplikasi atau dikembalikan ke keadaan awal menggunakan snapshot, sehingga eksperimen dapat diulang dengan hasil yang konsisten sebuah aspek penting dalam penelitian akademik.

Dalam penelitian ini, mesin virtual berfungsi sebagai wadah untuk seluruh komponen utama: VM attacker yang menghasilkan lalu lintas berbahaya, VM target yang menjalankan aplikasi web rentan, dan VM firewall yang mengimplementasikan Suricata serta modul klasifikasi XGBoost. Dengan menempatkan seluruh elemen penelitian dalam satu infrastruktur virtual, alur serangan SQL Injection dapat diamati dari awal hingga akhir, baik pada sisi aplikasi maupun pada sisi deteksi. Penggunaan mesin virtual memastikan bahwa seluruh

proses berjalan aman, terkontrol, dan tidak menimbulkan gangguan pada jaringan fisik.

2.2.9 Parameter Evaluasi

1. Confusion matrix

Confusion matrix adalah metode evaluasi dalam klasifikasi yang digunakan untuk menggambarkan performa model secara lebih rinci dibandingkan sekadar menghitung akurasi. Secara teknis, matriks ini terdiri dari empat nilai dasar, True Positive (TP), True Negative (TN), False Positive (FP), dan False Negative (FN).

Aktual/ Prediksi	Nilai Aktual	
	TP	FP
Nilai Prediksi	FN	TN

Keterangan:

TP: Nilai aktual positif yang benar diklasifikasikan sebagai positif.

FP: Nilai aktual negatif yang salah diklasifikasikan sebagai positif.

TN: Nilai aktual negatif yang benar diklasifikasikan sebagai negatif.

FN: Nilai aktual positif yang salah diklasifikasikan sebagai negatif.

Confusion matrix menjadi penting karena setiap komponennya merepresentasikan jenis hasil klasifikasi yang berbeda. Sebagaimana dapat diperhatikan dalam **tabel x.x**, TP menggambarkan kondisi ketika suatu sampel diklasifikasikan secara benar sebagai kelas positif. TN menunjukkan sampel yang diklasifikasikan secara benar sebagai kelas negatif. FP terjadi ketika suatu sampel sebenarnya termasuk kelas negatif, namun diklasifikasikan sebagai kelas positif oleh sistem. Sebaliknya, FN terjadi ketika sampel yang seharusnya termasuk kelas positif justru diklasifikasikan sebagai kelas negatif. Keempat komponen ini memberikan gambaran menyeluruh mengenai hasil prediksi model dan menjadi dasar untuk menilai karakteristik kinerja sistem klasifikasi.

Dalam konteks penelitian ini, confusion matrix digunakan untuk menilai kemampuan model XGBoost sebagai komponen keputusan kedua setelah Suricata. Dengan menganalisis distribusi TP, TN, FP, dan FN, dapat ditentukan apakah model memberikan manfaat nyata misalnya dengan meningkatkan tingkat deteksi

serangan SQL Injection atau mengurangi jumlah kesalahan deteksi yang biasanya terjadi pada pendekatan *signature*-based.

Selain matrix-matrix tersebut digunakan juga *False Positive Rate (FPR)* yang merupakan metrik evaluasi yang mengukur rasio kesalahan model dalam mengklasifikasikan data normal sebagai serangan. Dalam konteks sistem deteksi dan pencegahan intrusi, FPR merepresentasikan frekuensi munculnya *false alarm* yang dihasilkan oleh sistem. Secara matematis, FPR dihitung menggunakan persamaan berikut:

$$FPR = \frac{FP}{FP + TN}$$

Dimana *FP* merupakan jumlah nilai aktual benar yang diprediksi sebagai salah, dan *TN* merupakan jumlah nilai aktual benar yang memang diprediski sebagai benar.

Nilai FPR menjadi parameter krusial yang menentukan kualitas sistem dikarenakan mekanisme pencegahan pada IPS bersifat aktif, setiap aktivitas yang terdeteksi sebagai serangan akan otomatis diblokir sehingga nilai FPR tinggi juga menunjukkan potensi kesalahan deteksi benar yang terblokir, sehingga dapat secara langsung berdampak secara signifikan terhadap ketersediaan sistem.

Dari matriks tersebut, beberapa metrik turunan selanjutnya dihitung untuk memberikan gambaran yang lebih komprehensif, berupa *precision*, *recall*, *F1-score*, dan *Accuracy*. *Precision* mengukur seberapa tepat model dalam memutuskan bahwa sebuah sampel merupakan serangan, sedangkan *recall* mengukur seberapa banyak serangan yang benar-benar berhasil ditangkap oleh model. *F1-score* kemudian menggabungkan keduanya untuk memberikan satu nilai yang lebih seimbang terhadap kedua aspek.

2. Metrik Evaluasi

Berdasarkan nilai-nilai yang diperoleh dari confusion matrix, dilakukan perhitungan beberapa metrik evaluasi untuk menilai kinerja sistem secara lebih komprehensif. Metrik yang digunakan pada penelitian ini disesuaikan dengan karakteristik sistem, di mana kesalahan klasifikasi memiliki dampak langsung terhadap kredibilitas dan ketersediaan layanan, yang dinyatakan sebagai berikut:

$$Precision = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP + FN}$$

$$F1\text{-score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Metrik pertama adalah *precision*, yang digunakan untuk mengukur tingkat ketepatan sistem dalam mengklasifikasikan suatu permintaan sebagai serangan. Precision menunjukkan proporsi prediksi serangan yang benar terhadap seluruh prediksi serangan yang dihasilkan oleh sistem. Nilai precision yang tinggi menandakan bahwa sistem mampu menekan jumlah *false positive*, sehingga meminimalkan risiko pemblokiran terhadap lalu lintas yang sah.

Metrik kedua adalah *recall*, yang mengukur kemampuan sistem dalam mendeteksi seluruh serangan yang benar-benar terjadi. Recall menunjukkan proporsi serangan yang berhasil terdeteksi terhadap total serangan yang ada. Dalam konteks keamanan, nilai recall yang tinggi sangat penting karena *false negative* berpotensi menyebabkan serangan SQL Injection lolos dan mencapai aplikasi web target.

Untuk menyeimbangkan kedua metrik tersebut, digunakan F1-score sebagai metrik gabungan antara precision dan recall. F1-score memberikan gambaran performa sistem secara keseluruhan dengan mempertimbangkan trade-off antara ketepatan deteksi dan cakupan deteksi serangan. Metrik ini digunakan untuk menilai apakah sistem telah mencapai keseimbangan yang baik antara keamanan dan keberlangsungan layanan.

Selain itu, akurasi digunakan sebagai metrik pendukung untuk menunjukkan proporsi prediksi yang benar terhadap seluruh data yang diuji. Meskipun akurasi memberikan gambaran umum kinerja sistem, metrik ini tidak dijadikan satu-satunya acuan karena dapat menyesatkan pada kondisi distribusi data yang tidak seimbang antara lalu lintas normal dan serangan.

BAB 3 METODOLOGI

Bab ini menjelaskan metodologi yang digunakan dalam penyusunan serta pelaksanaan penelitian sesuai dengan tujuan yang telah ditetapkan pada bab sebelumnya. Metodologi diawali dengan pembahasan terkait metode penelitian yang digunakan kemudian dilanjutkan melalui serangkaian tahapan yang disusun secara terstruktur yang meliputi, studi literatur untuk memperkuat landasan teoritis, implementasi penelitian yang mencakup perancangan lingkungan pengujian, pengumpulan data, perancangan aturan Suricata, dan pelatihan model, serta skenario pengujian untuk mengevaluasi kinerja sistem yang dibangun. Tahapan akhir dari metodologi ini adalah pengambilan kesimpulan berdasarkan hasil evaluasi yang diperoleh. Seluruh alur metodologi penelitian digambarkan secara lebih jelas pada diagram alir pada gambar 3.1 sebagai pedoman dalam proses pelaksanaan penelitian.



Gambar 3.1 Diagram Alir Metode Penelitian

3.1 Metode Penelitian

Penelitian ini menggunakan pendekatan implementatif pengembangan, yaitu metode yang berfokus pada proses perancangan, pembangunan, dan evaluasi sistem secara langsung melalui serangkaian tahapan yang sistematis. Metode ini dipilih karena sesuai dengan tujuan penelitian, yaitu mengembangkan sebuah sistem *Hybrid Intrusion Prevention System*(HIPS) berbasis Suricata dan algoritma XGBoost untuk mitigasi serangan SQL Injection, sekaligus menguji efektivitasnya.

3.2 Studi Literatur

Tahap studi literatur pada penelitian ini dilakukan untuk memperoleh pemahaman yang menyeluruh mengenai konsep dan perkembangan terbaru dalam mitigasi serangan SQL Injection. Kajian dilakukan dengan memahami dasar-dasar keamanan aplikasi web serta mekanisme bagaimana serangan SQL Injection dapat terjadi.

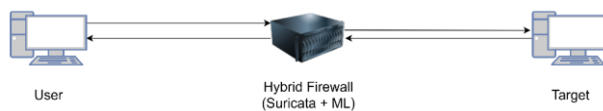
Peneliti (Tahun)	Fokus Penelitian	Temuan Utama
Abdullayev & Chauhan (2023)	Variasi teknik serangan SQL Injection (SQLi).	SQLi sangat dinamis dengan berbagai teknik (Union-based, Boolean-based, Time-based) yang dapat dimodifikasi penyerang.
Ouiazane dkk. (2022)	Mekanisme <i>Signature</i> -based Detection (Suricata).	Efektif dalam mendeteksi serangan melalui pencocokan pola (pattern matching) pada serangan yang polanya sudah diketahui.
Anggoro & Sulistyo (2019), Syamsuddin & Barukab (2022), Yunmar (2018)	Keterbatasan deteksi berbasis <i>signature</i> .	Sistem <i>signature</i> tidak cukup kuat menghadapi serangan modern yang menggunakan obfuscation, encoding, atau perubahan sintaksis.
Yogi (2025), De Carvalho Bertoli dkk. (2021), Pansare dkk. (2025)	Implementasi Machine Learning (XGBoost) & TF-IDF.	XGBoost unggul dalam mengenali pola kompleks melalui representasi numerik (TF-IDF) dan lebih adaptif dibanding sistem konvensional.

Literatur pada **tabel x.x** menunjukkan bahwa serangan ini memiliki banyak variasi teknik, mulai dari union-based hingga teknik yang lebih tersamarkan seperti boolean-based dan time-based. Variasi tersebut menggambarkan bahwa SQL Injection tidak hanya bergantung pada satu pola, tetapi dapat dimodifikasi sesuai kebutuhan penyerang (Abdullayev & Chauhan, 2023).

Kajian selanjutnya berfokus pada mekanisme kerja sistem deteksi berbasis *signature* seperti Suricata. Sistem ini bekerja dengan mencocokkan pola serangan yang sudah diketahui (Ouiazzane dkk., 2022). Meskipun efektif untuk serangan yang memiliki bentuk jelas, penelitian oleh Anggoro & Sulisty (2019), Syamsuddin & Barukab (2022) dan Yunmar (2018), menyoroti keterbatasan sistem *signature* ketika berhadapan dengan serangan yang telah mengalami perubahan sintaks, *obfuscation*, atau encoding tertentu. Kondisi ini menunjukkan bahwa deteksi berbasis *signature* saja tidak cukup untuk menangani variasi serangan modern yang semakin dinamis (Anggoro & Sulisty, 2019). Selain itu, studi literatur mencakup pemanfaatan machine learning dalam keamanan jaringan. Beragam penelitian menunjukkan bahwa algoritma gradient boosting seperti XGBoost memiliki kemampuan yang baik dalam mengenali pola serangan yang sulit dijangkau oleh sistem berbasis *signature* (Yogi, 2025, De Carvalho Bertoli dkk., 2021, dan Pansare dkk., 2025). XGBoost mampu memanfaatkan representasi numerik dari teks hasil ekstraksi fitur, termasuk TF-IDF, untuk membedakan *query* normal dari *query* yang berpotensi berbahaya. Pendekatan ini terbukti lebih adaptif karena model dapat belajar dari struktur data yang kompleks dan tidak bergantung pada pola yang sudah ditentukan sebelumnya.

Secara keseluruhan, literatur menunjukkan bahwa pendekatan *hybrid* yang menggabungkan sistem berbasis *signature* dan machine learning memiliki potensi besar dalam meningkatkan efektivitas deteksi SQL Injection. Temuan-temuan tersebut menjadi dasar bagi penelitian ini dalam merancang sistem yang memanfaatkan Suricata sebagai lapisan deteksi awal dan XGBoost sebagai analisis lanjutan untuk mengidentifikasi serangan yang tidak terdeteksi oleh *signature*. Pendekatan ini diharapkan dapat memberikan peningkatan yang signifikan terhadap akurasi dan ketahanan sistem dalam menghadapi variasi serangan.

3.3 Perancangan Sistem



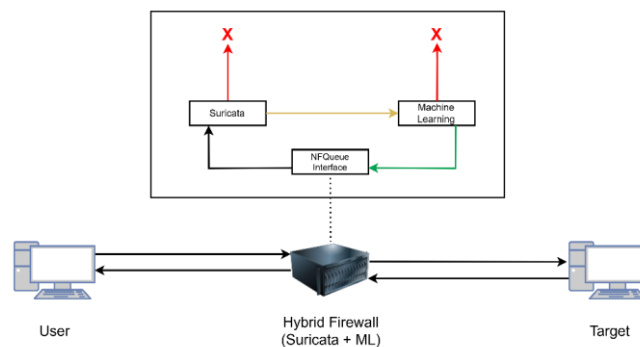
Gambar 3.2 Topologi Lingkungan Penelitian

Sebagaimana tampak pada Gambar 3.2, topologi sistem pada penelitian ini dirancang sebagai topologi inti yang menggambarkan alur pengamanan aplikasi web secara menyeluruh. Topologi terdiri atas tiga mesin virtual yang berjalan secara terpisah namun saling terhubung dalam satu lingkungan jaringan terkontrol. Penggunaan mesin virtual memungkinkan pengujian dilakukan secara terisolasi, konsisten, dan mudah direplikasi.

Mesin virtual pertama berperan sebagai VM user yang menghasilkan lalu lintas HTTP menuju aplikasi web, baik berupa permintaan normal maupun serangan SQL Injection. Seluruh lalu lintas dari VM user tidak dikirimkan langsung

ke server tujuan, melainkan diarahkan terlebih dahulu ke VM Hybrid Firewall. VM Hybrid Firewall merupakan komponen utama dalam sistem. Pada mesin virtual ini dijalankan mekanisme Intrusion Prevention System berbasis Suricata yang dikombinasikan dengan model machine learning XGBoost. Setiap permintaan yang masuk dianalisis untuk mendeteksi indikasi serangan. Permintaan yang terdeteksi sebagai serangan akan dihentikan, sedangkan permintaan yang dinilai aman diteruskan ke server aplikasi. Mesin virtual ketiga berperan sebagai VM target yang menjalankan layanan aplikasi web dan menjadi objek perlindungan sistem. Dengan susunan topologi ini, seluruh akses ke aplikasi web dipastikan melewati lapisan pengamanan terlebih dahulu. Topologi ini digunakan sebagai dasar perancangan sistem dan menjadi acuan konsisten untuk pembahasan pada tahap implementasi dan pengujian dalam penelitian ini.

3.3.1 Perancangan Lingkungan Penelitian



Commented [AN4]: Keknya perlu dimasukin bagian bagian dari gambar kesini deh?!

Lingkungan penelitian dibangun menggunakan arsitektur berbasis mesin virtual yang terdiri atas tiga komponen utama, yaitu *virtual machine*(VM) pengguna baik berupa penyerang maupun pengguna sah, VM firewall, dan VM *target*, sebagaimana ditunjukkan pada Gambar 3.2. VM attacker berfungsi sebagai sumber lalu lintas jaringan yang menghasilkan permintaan HTTP, baik berupa lalu lintas normal maupun trafik yang mengandung serangan SQL Injection. Permintaan tersebut ditujukan ke VM *target* yang menjalankan aplikasi web sasaran, namun seluruh lalu lintas terlebih dahulu diarahkan melalui VM firewall sebagai titik inspeksi dan pengambilan keputusan.

Pada VM firewall, proses pemeriksaan paket dilakukan secara berlapis. Lapisan pertama menggunakan Suricata sebagai sistem deteksi dan pencegahan intrusi berbasis *signature*. Suricata melakukan inspeksi awal terhadap paket HTTP dan akan langsung menjatuhkan paket yang sesuai dengan aturan serangan SQL Injection

yang telah didefinisikan, sebagaimana digambarkan oleh alur penolakan pada bagian Suricata pada Gambar 3.2. Paket yang tidak terdeteksi secara eksplisit oleh aturan *signature*, atau berada pada kondisi yang memerlukan analisis lanjutan, akan dialihkan ke antarmuka *Nfqueue*.

Nfqueue berperan sebagai penghubung antara mekanisme deteksi berbasis *signature* dan modul klasifikasi berbasis machine learning. Melalui antrean ini, paket diteruskan ke modul machine learning yang juga berjalan pada VM firewall. Pada tahap ini, *payload* HTTP diekstraksi dan ditransformasikan menjadi vektor fitur menggunakan metode TF-IDF, kemudian dianalisis oleh model XGBoost untuk menentukan apakah paket tersebut merupakan serangan SQL Injection atau permintaan normal. Hasil keputusan dari model dikembalikan melalui *Nfqueue* ke kernel untuk menentukan tindakan akhir terhadap paket, yaitu diteruskan (ACCEPT) atau dijatuhkan (DROP), sebagaimana ditunjukkan pada alur keputusan pada Gambar 3.2. Paket yang dinyatakan aman akan diteruskan ke VM *target* sehingga aplikasi web dapat memproses permintaan secara normal. Sebaliknya, paket yang diklasifikasikan sebagai serangan akan dihentikan pada VM firewall dan tidak pernah mencapai VM *target*.

3.3.2 Pengumpulan Data

Pengumpulan data pada penelitian ini dilakukan untuk memperoleh dua jenis *dataset*, yaitu *dataset* serangan SQL Injection dan *dataset* lalu lintas normal. *Dataset* serangan dihasilkan melalui simulasi serangan menggunakan alat pengujian Sqlmap (Begum dkk., 2025). Sqlmap dipilih karena merupakan salah satu alat yang paling umum digunakan untuk melakukan automasi serangan SQL Injection (Ikramaputra dkk., 2025), sehingga pola lalu lintas yang dihasilkan merepresentasikan skenario serangan yang realistis. Berikut merupakan jenis-jenis serangan yang digunakan dalam proses ini:

Table 3.1 Teknik Serangan SQLi pada Dataset

No	Teknik Serangan
1	Basic command Attack
2	Boolean based Attack
3	Error based Attack
4	Union based Attack
5	Time based Attack
6	Stacked query Attack
7	Multivariety level encoding Attack
8	SQLi tampering Attack (Single tamper Attack)
9	SQLi heavy tamper combo (multi tamper Attack)

Commented [AN5]:

Commented [A6]: Data source:
 Attack training: Sqlmap (redacted types)
 Benign training: CICIDS
 Attack testing: sqli dataset attack (Burpsuite)
 Benign testing: sql benign req (Burpsuite)

Commented [AN7]: Coba jelaskan si Sqlmap ini dibagian sql di 2.n

10	Random tamper & random user agent
11	Random user agent & delay

Seluruh lalu lintas yang dihasilkan selama proses simulasi direkam, kemudian dilakukan penyaringan dengan hanya mempertahankan permintaan HTTP yang benar-benar mengandung pola serangan SQL Injection.

Sementara itu, *dataset* lalu lintas normal diperoleh dari *dataset* publik CIC-IDS2017, dengan fokus pada bagian *dataset* yang memuat lalu lintas permintaan HTTP web normal. *Dataset* ini berisi berbagai permintaan HTTP, HTTP/2, HTTP/3, dan HTTPS ke halaman web dengan variasi parameter dan input pengguna yang sah serta tidak mengandung pola serangan. Pemilihan *dataset* ini bertujuan untuk menyediakan representasi lalu lintas normal yang beragam dan mendekati kondisi penggunaan aplikasi web pada lingkungan nyata. Kedua *dataset* tersebut kemudian melalui tahap pembersihan dan pengolahan awal.

Tahap pengumpulan data pelatihan diawali dengan pemilihan lalu lintas permintaan HTTP (*HTTP request*) yang bersumber dari aktivitas serangan dan normal. Setelah seluruh data terkumpul, dilakukan eliminasi terhadap data yang tidak relevan. Proses dilanjutkan dengan normalisasi data melalui penyeragaman komponen *host*, *path*, dan parameter non-kueri, serta penghapusan entri *user-agent*. Hal ini dilakukan untuk meminimalkan bias serta mencegah kecenderungan model dalam memberikan pembobotan berlebih terhadap data yang frekuensi kemunculannya tinggi.

Data yang telah diproses kemudian diklasifikasikan ke dalam dua label, yaitu serangan (*attack*) dan normal (*benign*). Selanjutnya, data tersebut ditransformasikan ke dalam representasi numerik melalui proses vektorisasi menggunakan metode TF-IDF yang diadaptasi dari teknik *Natural Language Processing* (NLP). Setelah tahap vektorisasi selesai, dataset serangan dan normal digabungkan serta diacak untuk menghindari bias urutan. Dataset akhir ini digunakan sebagai data pelatihan dan pengujian model inferensi XGBoost, yang berfungsi sebagai komponen pengambil keputusan pada sistem *Hybrid Intrusion Prevention System* yang dikembangkan dalam penelitian ini.

3.3.3 Perancangan Aturan Suricata

Ruleset Suricata pada penelitian ini dirancang dengan tujuan memblokir lalu lintas serangan SQL Injection, penerapan sistem deteksi berbasis *signature* umumnya bertumpu pada penggunaan basis aturan publik seperti aturan bawaan Suricata ataupun *Emerging Threats* (ET) *Open*. Ketergantungan terhadap aturan siap pakai

Commented [AN8]: Oke ini full ganti, jelasin gimana aturan dibuat.

tersebut justru memunculkan celah kegagalan yang cukup lebar, kerentanan ini mendasari arsitektur sistem untuk tidak mengadopsi *ruleset* bawaan secara mentah. Sebagai gantinya, sebanyak 24 aturan kustom dikonstruksi secara spesifik guna mengambil alih peran penapisan awal, sekaligus diposisikan sebagai *first-line static filter* yang mengunci lalu lintas berbahaya sebelum dialirkan ke lapisan analisis HIPS berikutnya.

Berikut adalah rangkaian proses pembuatan aturan kustom yang telah dibuat berdasarkan tahapan tahapan berikut ini:

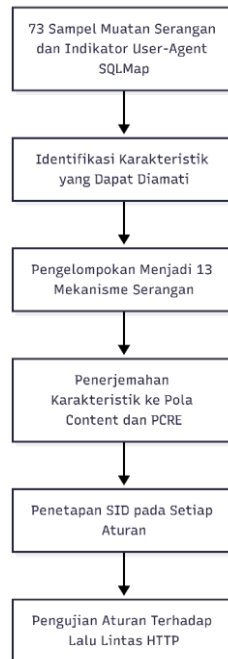
1. Pemetaan Vektor Serangan Berbasis Alur Kerja Sqlmap.

Tahap pertama dalam pengembangan aturan Suricata adalah memetakan vektor serangan SQL Injection berdasarkan alur kerja Sqlmap. Tahap ini bertujuan mengidentifikasi pola serangan yang dapat diamati pada lalu lintas HTTP, mengelompokkan pola tersebut berdasarkan mekanisme serangannya, dan menerjemahkannya menjadi aturan deteksi yang memiliki ID *signature* atau *Signature ID* (SID).

Sqlmap dipilih sebagai acuan operasional karena mampu menjalankan pengujian SQL Injection secara otomatis dan sistematis. Sqlmap tidak hanya mengirimkan pola injeksi dasar, tetapi juga menjalankan beberapa tahapan pengujian, seperti mengidentifikasi jenis sistem manajemen basis data, menguji titik injeksi, menjalankan eksploitasi berbasis respons, melakukan enumerasi informasi basis data, serta menerapkan teknik penyamaran muatan. Kemampuan tersebut membuat Sqlmap dapat menghasilkan variasi serangan yang lebih luas dibandingkan dengan pengujian manual menggunakan satu atau dua pola injeksi (Q. Li dkk., 2024).

Alur kerja Sqlmap yang dijadikan dasar pemetaan mencakup identifikasi alat, pengujian sintaks dasar, eksploitasi berbasis kondisi, eksploitasi berbasis waktu (*time-based*), pengambilan data, identifikasi DBMS, enumerasi struktur basis data, penyamaran *query*, dan pengujian akses. Setiap tahapan menghasilkan karakteristik tertentu pada bagian URI, parameter URL, badan permintaan HTTP, atau *header* HTTP. Karakteristik tersebut digunakan sebagai titik pengawasan dalam penyusunan aturan Suricata.

Secara ringkas, proses pemetaan dilakukan melalui alur berikut:



Sebanyak 73 sampel muatan digunakan untuk mewakili 12 mekanisme utama SQL Injection. Selain pola yang terdapat pada muatan, identitas SQLMap pada *header* User-Agent digunakan sebagai indikator mekanisme ke-13. Pemisahan tersebut dilakukan karena deteksi User-Agent tidak hanya memeriksa sintaks SQL di dalam parameter, tetapi mengidentifikasi penggunaan alat otomatis berdasarkan informasi yang dikirim pada *header* HTTP.

Aturan-aturan yang dirumuskan mencakup kepada keseluruhan jenis muatan pada tabel (x.x) berikut ini:

No	Jenis Serangan	Jenis Pengawasan	Karakteristik	SID	Jumlah Aturan
1	Identifikasi alat	Deteksi agent pengujian	Kata sqlmap pada <i>header</i> User-Agent HTTP	1000001	1
2	Serangan Umum	Pola dasar SQL Injection	Tanda petik tunggal, tanda petik ganda, dan penanda komentar SQL pada parameter URL	1000002–1000003	2

3	<i>Blind-Sql</i>	<i>Boolean-based blind SQL Injection</i>	Pola kondisi logika seperti AND 1=1 dan OR 2=2 pada permintaan GET atau POST	1000004–1000005	2
4	<i>Blind-Sql</i>	<i>Time-based blind SQL Injection</i>	Fungsi SLEEP(), pg_sleep(), dan perintah WAITFOR DELAY	1000006–1000009	4
5	Ekstraksi data	<i>UNION-based SQL Injection</i>	Fungsi EXTRACTVALUE() dan kombinasi CONCAT dengan nilai 0x7e	1000010–1000011	2
6	Eksplorasi error	<i>Error-based SQL Injection</i>	Fungsi EXTRACTVALUE() dan kombinasi CONCAT dengan nilai 0x7e	1000012–1000013	2
7	Samaran Muatan query	<i>Encoding URL</i>	Karakter SQL yang diubah ke bentuk URL <i>encoding</i> , seperti %27, %22, %23, dan %2D%2D	1000014	1
8	Identifikasi DBMS	Fingerprinting basis data	Penggunaan @@version, version(), dan @@SERVERNAME	1000015–1000017	3
9	Enumerasi struktur	Pengujian jumlah kolom	Pola ORDER BY yang diikuti indeks numerik	1000018	1
10	Samaran Muatan query	<i>Null byte injection</i>	Penggunaan karakter null dalam bentuk %00	1000019	1
11	Pengumpulan informasi	Fungsi informasi basis data	Penggunaan CURRENT_USER(), USER(), SESSION_USER(), DATABASE(), dan SCHEMA()	1000020–1000021	2
12	Samaran Muatan query	Pengodean heksadesimal	Nilai SQL dalam bentuk heksadesimal yang diawali dengan 0x	1000023	1
13	Percobaan akses	Akses berkas pada server basis data	Penggunaan fungsi LOAD_FILE()	1000024	1

Pada tabel (x.x) diatas, dijabarkan bahwa satu mekanisme serangan dapat diterjemahkan menjadi beberapa aturan apabila memiliki konteks pemeriksaan yang berbeda. Serangan berbasis Boolean dan UNION, misalnya, menggunakan aturan terpisah untuk permintaan GET dan POST. Pemisahan juga diterapkan pada serangan berbasis waktu karena setiap DBMS memiliki fungsi penundaan yang berbeda, yaitu SLEEP() pada MySQL, pg_sleep() pada PostgreSQL, dan WAITFOR DELAY pada SQL Server. Hubungan antara muatan serangan dan SID tidak bersifat satu banding satu. Satu SID dapat mendeteksi beberapa variasi muatan yang memiliki pola inti serupa, sedangkan satu muatan dapat memicu lebih dari satu aturan apabila memuat beberapa karakteristik sekaligus, seperti UNION SELECT, penanda komentar, dan karakter yang telah dikodekan dalam format URL.

Pemetaan dilakukan dengan mengidentifikasi pola yang konsisten pada setiap variasi muatan, meliputi kata kunci SQL, fungsi khusus DBMS, operator logika, penanda komentar, pola pengodean, serta posisi pola pada URI, parameter URL, *body request*, metode HTTP, atau *header* User-Agent. Pola yang terlalu umum tidak langsung digunakan karena berpotensi menimbulkan kesalahan deteksi pada lalu lintas normal. Pada tahap implementasi, pola sederhana diperiksa menggunakan kata kunci content, sedangkan pola yang lebih kompleks diperiksa menggunakan Perl Compatible Regular Expression (PCRE). Opsi *nocase* digunakan agar pencocokan tidak membedakan huruf kapital dan huruf kecil, sedangkan *flowbits* digunakan untuk menandai aliran yang telah terdeteksi agar peringatan yang sama tidak muncul berulang kali dalam satu aliran komunikasi.

2. Kategorisasi Aturan

Untuk mempermudah analisis, aturan Suricata tidak disajikan sebagai daftar terpisah, tetapi dikelompokkan ke dalam lima klaster fungsional. Pengelompokan ini didasarkan pada tujuan setiap aturan dalam alur serangan SQL Injection, mulai dari identifikasi awal, pengujian titik injeksi, eksploitasi lanjutan, penyamaran muatan, hingga percobaan mengakses sumber daya pada sistem basis data. Pendekatan tersebut menunjukkan bahwa aturan yang dikembangkan tidak hanya mendeteksi sintaks SQL secara umum, tetapi juga mencakup pola khusus DBMS dan teknik penghindaran deteksi.

No	Klaster Fungsional	SID	Jumlah	Fokus Deteksi
1	Reconnaissance and Fingerprinting	1000001, 1000015– 1000018,	7	Mendeteksi penggunaan User-Agent bawaan SQLMap, kueri identifikasi DBMS,

		1000020– 1000021		pengujian jumlah kolom, serta pengambilan informasi pengguna, basis data, dan skema.
2	Syntactic and Basic Injection	1000002, 1000004– 1000005	3	Mendeteksi tanda kutip sebagai pemutus sintaks kueri dan pola logika Boolean eksplisit pada permintaan GET maupun POST.
3	Advanced and Blind Exploitation	1000006– 1000013	8	Mendeteksi eksploitasi berbasis waktu, UNION, dan galat melalui fungsi khusus MySQL, PostgreSQL, dan SQL Server.
4	Obfuscation and Evasion Handling	1000003, 1000014, 1000019, 1000023	4	Mendeteksi penanda komentar, pengodean URL, penyisipan null byte, dan representasi heksadesimal yang digunakan untuk menyamarkan muatan.
5	High-Risk System Access	1000024	1	Mendeteksi penggunaan fungsi LOAD_FILE() yang dapat digunakan untuk mencoba membaca berkas pada sistem server basis data.

Dalam Tabel (x.x), setiap klaster disusun untuk mewakili sasaran deteksi dari aturan Suricata yang telah dikembangkan. Masing-masing kategori mencakup fokus pemeriksaan yang berbeda, mulai dari aktivitas pengenalan awal terhadap sistem basis data, pola injeksi dasar, eksploitasi lanjutan, teknik penyamaran muatan, hingga upaya akses terhadap sumber daya sistem. Pengelompokan ini memudahkan analisis terhadap cakupan fungsi setiap aturan serta menunjukkan posisi masing-masing *signature* dalam tahapan serangan SQL Injection.

Klaster Reconnaissance and Fingerprinting mencakup aturan yang digunakan untuk mengenali aktivitas awal sebelum eksploitasi dilakukan. Aktivitas tersebut meliputi deteksi identitas SQLMap melalui *header* User-Agent, pencarian versi DBMS menggunakan @@version, version(), atau @@SERVERNAME, pengujian jumlah kolom melalui ORDER BY, serta pengambilan

informasi menggunakan `CURRENT_USER()`, `USER()`, `DATABASE()`, dan `SCHEMA()`. Pola-pola tersebut menunjukkan adanya upaya untuk mengenali jenis, struktur, dan konteks basis data yang menjadi target.

Klaster Syntactic and Basic Injection memuat pola awal yang digunakan untuk menguji apakah masukan pengguna dapat memengaruhi sintaks kueri. Aturan pada klaster ini memeriksa tanda kutip tunggal atau ganda serta ekspresi Boolean seperti `AND 1=1` dan `OR 1=1`. Pemisahan aturan GET dan POST dilakukan karena lokasi muatan berada pada bagian permintaan HTTP yang berbeda.

Klaster Advanced and Blind Exploitation mencakup teknik yang digunakan setelah titik injeksi berhasil ditemukan. Aturan berbasis waktu memeriksa fungsi `SLEEP()`, `pg_sleep()`, dan `WAITFOR DELAY` untuk mendeteksi serangan blind SQL Injection. Aturan lainnya memeriksa `UNION SELECT` untuk mendeteksi upaya penggabungan hasil kueri serta `EXTRACTVALUE()` dan `CONCAT` dengan nilai `0x7e` untuk mendeteksi manipulasi pesan galat.

Klaster Obfuscation and Evasion Handling berfungsi menangkap perubahan bentuk muatan yang bertujuan menghindari pencocokan *signature* langsung. Teknik yang diperiksa meliputi penanda komentar SQL, karakter yang dikodekan dalam format URL, penyisipan null byte, dan representasi heksadesimal. Walaupun bentuk muatannya berubah, pola inti serangan tetap dapat dikenali melalui kombinasi pemeriksaan content dan PCRE.

Klaster High-Risk System Access ditempatkan secara terpisah karena pola yang dideteksi memiliki dampak lebih tinggi dibandingkan pengujian sintaks biasa. Fungsi `LOAD_FILE()` dapat digunakan untuk mencoba membaca berkas yang dapat diakses oleh layanan basis data. Oleh karena itu, kemunculan fungsi tersebut pada parameter HTTP diperlakukan sebagai indikator upaya akses terhadap sumber daya sistem.

Kelima klaster tersebut tidak selalu menggambarkan tahapan serangan yang berlangsung secara berurutan. Satu permintaan dapat memuat pola dari beberapa klaster sekaligus. Sebagai contoh, muatan `UNION SELECT` yang dikodekan dalam format URL dapat memicu aturan Advanced and Blind Exploitation serta Obfuscation and Evasion Handling. Kategorisasi ini digunakan untuk menjelaskan cakupan fungsional ruleset dan memudahkan analisis terhadap jenis serangan yang terdeteksi.

Hasil tahap ini berupa pemetaan yang menghubungkan alur serangan SQLMap, karakteristik muatan, bagian lalu lintas HTTP yang diperiksa, dan SID Suricata. Pemetaan tersebut menjadi dasar untuk tahap pengembangan aturan karena setiap *signature* dibentuk dari pola yang telah ditemukan pada sampel serangan. Pendekatan ini membuat proses penyusunan aturan lebih terstruktur dan dapat ditelusuri, bukan hanya didasarkan pada pemilihan kata kunci SQL secara acak.

3. Konstruksi Sintaksis Aturan dan Alokasi Buffer

Aturan Suricata dibentuk oleh tiga komponen utama, yaitu *Action*, *header*, dan opsi pemeriksaan. Komponen tindakan menentukan respons Suricata ketika pola serangan ditemukan. Pada tahap pengembangan dan pengujian awal, aturan masih menggunakan mode peringatan sehingga lalu lintas hanya dicatat sebagai kejadian keamanan tanpa dihentikan. Setelah aturan selesai divalidasi, tindakan tersebut diubah menjadi pemblokiran agar Suricata dapat berfungsi sebagai sistem pencegahan. Penerapan pemblokiran dilakukan dalam mode inline melalui NFQUEUE sehingga setiap permintaan diperiksa sebelum diteruskan menuju server. Header aturan menetapkan protokol, arah komunikasi, serta cakupan alamat dan port yang diperiksa. Pada lingkungan pengujian terkontrol, alamat dan port sumber maupun tujuan tidak dibatasi agar seluruh lalu lintas HTTP yang melewati Suricata dapat dianalisis.

Komponen	Fungsi teknis
<i>Action Rule</i>	Menentukan apakah kejadian hanya dicatat atau langsung diblokir.
<i>Header Traffic</i>	Menentukan protokol, <i>flow</i> , <i>address</i> , dan port yang diperiksa.
<i>Buffer HTTP</i>	Membatasi pemeriksaan pada URI, <i>body request</i> , atau <i>header</i> tertentu.
Pencocokan pola	Mendeteksi kata kunci dan struktur <i>query</i> SQL Injection.
Pencocokan tanpa sensitivitas huruf	Mendeteksi variasi huruf kapital dan huruf kecil pada pola serangan.
NFQUEUE	Meneruskan lalu lintas ke Suricata untuk diperiksa dan diberi keputusan sebelum mencapai server.

Pemeriksaan pola dilakukan pada buffer HTTP yang sesuai dengan lokasi *query*. Parameter yang dikirim melalui metode GET diperiksa pada bagian URI, sedangkan muatan pada metode POST diperiksa pada *body request*. Informasi identitas perangkat lunak, seperti User-Agent, diperiksa secara khusus pada *header* terkait. Penempatan pola pada buffer tertentu membuat pemeriksaan

lebih terarah dan mengurangi kemungkinan lalu lintas normal terdeteksi sebagai serangan. Aturan juga menerapkan pencocokan tanpa membedakan huruf kapital dan huruf kecil untuk mengantisipasi variasi penulisan kata kunci SQL. Pola sederhana diperiksa melalui pencocokan langsung, sedangkan pola yang memiliki susunan lebih kompleks diperiksa menggunakan ekspresi reguler agar struktur serangan dapat dikenali secara lebih spesifik.

3.3.4 Pelatihan Model Pembelajaran Mesin

Proses pelatihan dilakukan dengan menggunakan algoritma XGBoost pada *dataset* yang telah disusun. Sebelum dilakukan pelatihan model, seluruh data teks berupa *query* HTTP diproses menggunakan metode *Term Frequency–Inverse Document Frequency* (TF-IDF). TF-IDF digunakan untuk mengubah data berbasis teks menjadi representasi numerik sehingga dapat diproses oleh algoritma XGBoost. Melalui mekanisme pembobotan TF-IDF, istilah atau simbol yang memiliki karakteristik khas serangan SQL Injection akan memperoleh bobot lebih tinggi, sementara istilah umum yang sering muncul pada lalu lintas normal akan memiliki bobot yang lebih rendah.

Pemilihan algoritma XGBoost didasarkan pada penelitian sebelumnya yang menunjukkan bahwa kombinasi TF-IDF dan XGBoost memiliki kinerja yang sangat baik dalam mendeteksi serangan SQL Injection berbasis teks. Penelitian oleh Pansare dkk, (2025). melaporkan bahwa pendekatan ini mampu mencapai tingkat akurasi yang tinggi sekaligus menjaga keseimbangan antara precision dan recall dalam proses klasifikasi *query* SQL. Temuan tersebut menjadi dasar pemilihan XGBoost sebagai model inferensi utama pada penelitian ini. Selama proses pelatihan, sejumlah parameter penting pada XGBoost diatur melalui proses pencarian terstruktur, meliputi nilai *learning rate*, *max depth*, jumlah pohon keputusan, serta parameter regularisasi. Pengaturan parameter ini dilakukan untuk memperoleh model dengan kemampuan generalisasi yang baik dan meminimalkan risiko overfitting. Selain itu, untuk mengatasi potensi ketidakseimbangan kelas antara data serangan dan data normal, diterapkan penyesuaian bobot kelas agar model tidak cenderung memihak pada kelas mayoritas.

Model yang telah dilatih kemudian diintegrasikan dengan sistem Suricata melalui mekanisme *NFQUEUE*. Dengan integrasi ini, setiap paket yang tidak terdeteksi oleh aturan Suricata dapat diteruskan ke modul TF-IDF dan XGBoost untuk dianalisis lebih lanjut secara real-time. Pendekatan ini memungkinkan sistem *hybrid* memberikan keputusan yang lebih adaptif terhadap variasi serangan SQL Injection

yang tidak dapat ditangani sepenuhnya oleh mekanisme berbasis *signature*.

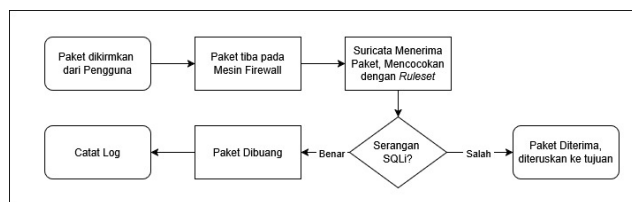
3.4 Skenario Pengujian dan Evaluasi

3.4.1 Skenario Pengujian

Skenario pengujian pada penelitian ini dirancang untuk mengevaluasi kinerja sistem dalam mendeteksi dan mencegah serangan SQL Injection berdasarkan pendekatan yang digunakan. Pengujian dilakukan melalui tiga skenario utama, yaitu pengujian menggunakan Suricata saja, pengujian menggunakan model machine learning saja, dan pengujian menggunakan sistem *hybrid* yang menggabungkan Suricata dan XGBoost. Pembagian skenario ini bertujuan untuk memperoleh perbandingan kinerja yang objektif antara masing-masing pendekatan, sekaligus menilai kontribusi setiap komponen dalam sistem yang dikembangkan.

Skenario pengujian akan dilakukan dengan membandingkan tiga pendekatan IPS, yakni:

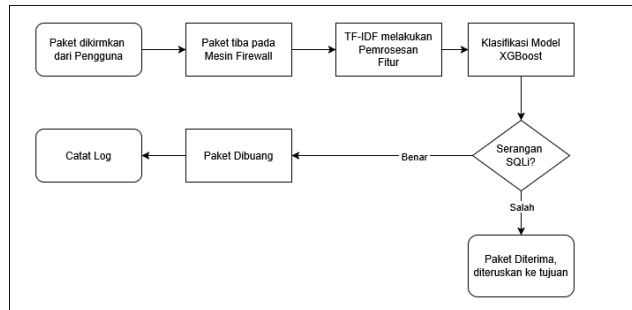
1. Skenario Pengujian Suricata Tunggal.



[diagram scenario 1 based on gambar 3.2]

Pada skenario pertama, pengujian dilakukan dengan mengaktifkan Suricata sebagai satu-satunya mekanisme deteksi dan pencegahan. Seluruh lalu lintas HTTP yang mengandung serangan SQL Injection diuji terhadap aturan Suricata yang telah dirancang. Hasil pengujian pada skenario ini digunakan untuk mengukur efektivitas deteksi berbasis *signature*, termasuk kemampuan Suricata dalam mengenali pola serangan yang eksplisit serta keterbatasannya dalam menghadapi variasi serangan yang telah mengalami obfuscation atau modifikasi sintaks.

2. Skenario Pengujian *Machine Learning* Tunggal.



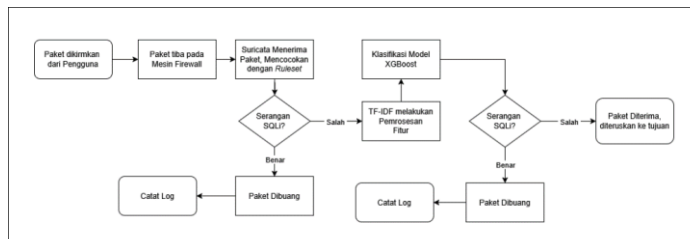
[diagram scenario 2 based on gambar 3.2]

Skenario kedua difokuskan pada pengujian model machine learning secara terpisah, tanpa melibatkan mekanisme deteksi berbasis *signature*. Pada skenario ini, seluruh permintaan HTTP yang masuk diproses menggunakan pipeline TF-IDF dan diklasifikasikan oleh model XGBoost. Pengujian ini bertujuan untuk mengevaluasi kemampuan model dalam mengenali pola serangan SQL Injection berdasarkan karakteristik data, sekaligus mengukur tingkat false positive dan false negative ketika model digunakan sebagai satu-satunya pengambil keputusan.

Commented [AN9]: Masukan ekspektasi outputnya selain tujuan

Commented [AN10R9]: Sama buat tiap skenario

3. Skenario Pengujian Pendekatan Sistem *Hybrid*.



[diagram scenario 3 based on gambar 3.2]

Skenario ketiga merupakan pengujian sistem *hybrid* yang mengombinasikan Suricata dan model machine learning. Dalam skenario ini, Suricata berfungsi sebagai lapisan deteksi pertama, sedangkan paket yang tidak terdeteksi atau berada pada kondisi ambigu diteruskan ke modul TF-IDF dan XGBoost melalui mekanisme *NFQUEUE*. Keputusan akhir terhadap paket ditentukan berdasarkan hasil analisis kedua lapisan tersebut. Pengujian pada skenario ini digunakan untuk menilai peningkatan efektivitas sistem secara keseluruhan dibandingkan dengan penggunaan Suricata atau machine learning secara terpisah.

Seluruh pengujian dilakukan menggunakan dua metode serangan, yaitu serangan otomatis menggunakan alat Sqlmap dan

pengujian manual yang dilakukan oleh **penguji terkontrol**. Pengujian manual dilakukan dengan mensimulasikan serangan SQL Injection secara langsung ke sistem dalam kurun waktu 24 jam. Waktu pengujian bersifat fleksibel namun tetap terukur, dengan seluruh aktivitas dilakukan pada jaringan terisolasi untuk mendapatkan hasil yang maksimal dan tanpa intervensi eksternal.

Commented [AN11]: Harus di list di latbel

Commented [AN12R11]: Bikin kerangka pengujian, jelasin gimana scenario bakal dilakukan

3.4.2 Parameter Evaluasi

Pada tahap evaluasi kinerja sistem, digunakan confusion matrix sebagai dasar utama untuk mengukur performa deteksi dan pencegahan serangan SQL Injection pada setiap skenario pengujian. Evaluasi dilakukan secara terpisah untuk tiga skenario, yaitu pengujian Suricata saja, pengujian model machine learning saja, dan pengujian sistem hybrid. Pendekatan ini memungkinkan perbandingan yang objektif terhadap efektivitas masing-masing metode serta kontribusinya dalam sistem yang dikembangkan.

Confusion matrix terdiri atas empat komponen, yaitu *True Positive* (TP), *True Negative* (TN), *False Positive* (FP), dan *False Negative* (FN). Komponen ini digunakan untuk menghitung metrik evaluasi yang relevan dalam konteks Intrusion Prevention System, meliputi *precision*, *recall*, *F1-score*, dan akurasi. *Precision* digunakan untuk menekan kesalahan *false positive* yang berpotensi mengganggu lalu lintas sah, sedangkan *recall* digunakan untuk meminimalkan *false negative* agar serangan tidak lolos dari sistem. *F1-score* digunakan untuk menilai keseimbangan antara kedua metrik tersebut, sementara akurasi memberikan gambaran umum kinerja sistem.

3.5 Pengambilan Kesimpulan

Kesimpulan ditarik berdasarkan hasil pengujian pada seluruh skenario, yaitu pengujian menggunakan Suricata saja, pengujian menggunakan model machine learning saja, dan pengujian menggunakan sistem hybrid. Setiap hasil pengujian dianalisis menggunakan parameter evaluasi berbasis confusion matrix untuk memastikan penilaian dilakukan secara objektif dan terukur.

Kesimpulan penelitian difokuskan pada perbandingan efektivitas masing-masing pendekatan dalam mendeteksi dan mencegah serangan SQL Injection, serta pada kemampuan sistem hybrid dalam meningkatkan kinerja deteksi dibandingkan dengan pendekatan tunggal. Selain itu, analisis juga mempertimbangkan dampak kesalahan klasifikasi, khususnya false positive dan false negative, terhadap keamanan dan ketersediaan layanan aplikasi web.

BAB 4 IMPLEMENTASI

4.1 Implementasi Lingkungan Penelitian

Subbab ini memaparkan spesifikasi teknis dari infrastruktur yang digunakan untuk menjalankan sistem.

4.1.1 Spesifikasi Perangkat Keras

Rincian spesifikasi mesin fisik atau virtual (*testbed*) yang digunakan untuk menjalankan Suricata, modul *machine learning*, dan simulasi penyerangan (misalnya, spesifikasi CPU, RAM, dan arsitektur mesin).

4.1.2 Spesifikasi Perangkat Lunak

Daftar sistem operasi, pustaka (*library*), bahasa pemrograman (Python/C++), dan versi utilitas yang terlibat (seperti versi Suricata, pustaka XGBoost, dan scikit-learn untuk TF-IDF).

4.2 Implementasi Infrastruktur dan Topologi Jaringan

Mendeskripsikan pengaturan jaringan secara praktis. Anda perlu menjelaskan konfigurasi antarmuka jaringan (*network interface*), penempatan mesin penyerang (*attacker*), dan mesin target (*web server/database*). Bagian ini memastikan penguji memahami bahwa paket lalu lintas SQL Injection mengalir melalui titik inspeksi yang tepat sebelum mencapai basis data.

4.3 Pengumpulan Data

Bagian ini menguraikan tahapan mekanis dalam merekayasa dan merekam lalu lintas jaringan untuk membentuk *dataset* pelatihan dan pengujian.

4.3.1 Pengumpulan Data Serangan SQL Injection

Deskripsi eksekusi otomatisasi serangan. Anda perlu memuat cuplikan parameter atau skrip yang digunakan (misalnya perintah baris SQLMap) untuk membombardir *web server* dengan berbagai muatan SQLi (*error-based*, *time-based*, *boolean*), serta proses perekaman paket tersebut menjadi format PCAP/CSV.

4.3.2 Pengumpulan Data Lalu Lintas Normal

Penjelasan teknis mengenai skrip pembangkit lalu lintas aman (*benign traffic*). Bagian ini memaparkan bagaimana interaksi pengguna sah disimulasikan (seperti pengisian formulir login secara normal) untuk menghasilkan data negatif (*non-malicious*) penyeimbang *dataset*.

4.4 Implementasi Ruleset Suricata

Fokus pada integrasi aturan kustom ke dalam mesin penapis. Subbab ini menjelaskan modifikasi pada berkas `suricata.yaml` (pengaturan *mode*

inline/IPS), proses pemuatan 24 aturan kustom SQLi yang dirancang di Bab 3 ke dalam sistem, dan konfigurasi pendelegasian paket menggunakan NFQUEUE.

4.5 Implementasi Model

Menguraikan transformasi matematis dan komputasi *machine learning* terhadap paket yang ditangkap.

4.5.1 Implementasi Model TF-IDF

Potongan kode dan penjelasan fungsi yang mengubah teks muatan HTTP mentah hasil ekstraksi menjadi representasi matriks vektor (TF-IDF). Ini adalah proses standardisasi masukan sebelum dikonsumsi oleh algoritma klasifikasi.

4.5.2 Implementasi Model XGBoost

Implementasi logika algoritma *Extreme Gradient Boosting*. Berisi deklarasi parameter pelatihan (*hyperparameter*), inisialisasi model, dan mekanisme eksekusi keputusan. Anda juga perlu memasukkan logika *Action Engine* di sini: bagaimana probabilitas prediksi model diterjemahkan menjadi perintah lapisan jaringan (`DROP` untuk serangan, `ACCEPT` untuk lalu lintas normal).

4.6 Pengujian dan Evaluasi

Subbab penutup implementasi yang mendeskripsikan metrik dan skenario pengujian yang akan dilakukan. Bagian ini merinci parameter *Confusion Matrix* (True Positive, False Positive, dll) dan rumusan evaluasi kinerja sistem yang mencakup akurasi, presisi, *recall*, *F1-Score*, serta pengujian beban/latensi pemrosesan paket.

BAB 5 HASIL DAN ANALISIS

5.1 Hasil

Subbab ini menyajikan data empiris dan bukti fungsional dari serangkaian pengujian tanpa menyertakan interpretasi mendalam. Fakta disajikan secara langsung melalui tabel, grafik, dan log sistem.

5.1.1 Hasil Implementasi *hybrid* IPS

Menyajikan bukti visual dan log terminal yang memvalidasi operasional sistem secara *end-to-end*. Bagian ini memuat tangkapan layar (atau potongan teks log) saat paket HTTP ditangkap oleh aturan kustom Suricata, proses pendelegasian paket yang lolos menuju antarmuka NFQUEUE, ekstraksi fitur TF-IDF oleh skrip Python, hingga keluaran putusan akhir dari model XGBoost berupa penjatuhan paket (*DROP*) atau penerusan paket (*ACCEPT*).

5.1.2 Hasil Kinerja IPS

Memaparkan rekapitulasi data kuantitatif dari pengujian performa deteksi. Bagian ini berisi tabel *Confusion Matrix* (berisi nilai *True Positive*, *True Negative*, *False Positive*, dan *False Negative*) untuk tiga konfigurasi pengujian: Suricata tunggal, model XGBoost tunggal, dan *hybrid* IPS. Selanjutnya, sajikan tabel konversi dari matriks tersebut menjadi persentase Akurasi, Presisi, *Recall*, dan *F1-Score*.

5.2 Analisis

Subbab ini membongkar data numerik pada subbab 5.1 untuk menghasilkan argumentasi ilmiah. Fokus utamanya adalah menjawab rumusan masalah dengan menjelaskan justifikasi mekanis di balik angka-angka tersebut.

5.2.1 Analisis Efektifitas Kinerja IPS

Menguraikan rasionalisasi di balik angka metrik kinerja arsitektur hibrida. Bagian ini menjelaskan secara teknis bagaimana mesin klasifikasi (TF-IDF dan XGBoost) bertindak sebagai jaring pengaman lapis kedua, serta membahas insiden spesifik terkait paket normal yang mungkin salah diblokir (*False Positive*) atau paket serangan mutakhir yang tetap lolos (*False Negative*) dari arsitektur *hybrid* ini.

5.2.2 Analisis Perbandingan Kinerja IPS

Membahas komparasi fungsional antara skenario Suricata tunggal, XGBoost tunggal, dan arsitektur hibrida. Argumentasi di sini difokuskan pada kelemahan penapisan statis Suricata saat menghadapi teknik *obfuscation* atau manipulasi sintaks, yang kemudian berhasil ditutupi oleh fleksibilitas algoritma *machine learning*. Analisis ini bertugas membuktikan secara logis bahwa

penggabungan keduanya (HIPS) menghasilkan performa deteksi SQL Injection yang paling optimal dibandingkan jika masing-masing sistem berdiri sendiri.

BAB 6 PENUTUP

6.1 Kesimpulan

6.2 Saran

DAFTAR PUSTAKA

14. *Setting up IPS/inline for Linux — Suricata 6.0.14 documentation* (tanpa tanggal). Tersedia pada: <https://docs.suricata.io/en/suricata-6.0.14/setting-up-ipsinline-for-linux.html> (Diakses: 25 September 2025).

160 "Cybersecurity Statistics: Updated Report 2025" (2022), 19 Desember. Tersedia pada: <https://www.getastra.com/blog/security-audit/cyber-security-statistics/> (Diakses: 23 September 2025).

Agoramoorthy, M. dkk. (2023) "An Analysis of Signature-Based Components in Hybrid Intrusion Detection Systems," dalam *2023 Intelligent Computing and Control for Engineering and Business Systems (ICCEBS). 2023 Intelligent Computing and Control for Engineering and Business Systems (ICCEBS)*, Chennai, India: IEEE, hlm. 1–5. Tersedia pada: <https://doi.org/10.1109/ICCEBS58601.2023.10449209>

Alkhudhayr, F. dkk. (2019) "Information Security: A Review of Information Security Issues and Techniques," dalam *2019 2nd International Conference on Computer Applications & Information Security (ICCAIS). 2019 2nd International Conference on Computer Applications & Information Security (ICCAIS)*, Riyadh, Saudi Arabia: IEEE, hlm. 1–6. Tersedia pada: <https://doi.org/10.1109/CAIS.2019.8769504>.

Alwan, Z. dan Younis, M. (2017) "Detection and Prevention of SQL Injection Attack: A Survey," *International Journal of Computer Science and Mobile Computing*, 68, hlm. 5–17

Aslan, O. dan Samet, R. (2020) "A Comprehensive Review on Malware Detection Approaches," *IEEE Access*, 8, hlm. 6249–6271. Tersedia pada: <https://doi.org/10.1109/ACCESS.2019.2963724>

Carter, P.A. (2018) "SQL Injection," dalam *Securing SQL Server: DBAs Defending the Database*. Berkeley, CA: Apress, hlm. 221–245. Tersedia pada: https://doi.org/10.1007/978-1-4842-4161-5_10.

De Carvalho Bertoli, G. dkk. (2021) "An End-to-End Framework for Machine Learning-Based Network Intrusion Detection System," *IEEE Access*, 9, hlm. 106790–106805. Tersedia pada: <https://doi.org/10.1109/ACCESS.2021.3101188>

Guo, Y. (2023) "A review of Machine Learning-based zero-day attack detection: Challenges and future directions," *Computer Communications*, 198, hlm. 175–185. Tersedia pada: <https://doi.org/10.1016/j.comcom.2022.11.001>

Gupta, N., Jindal, V. dan Bedi, P. (2023) "A Survey on Intrusion Detection and Prevention Systems," *SN Computer Science*, 4(5), hlm. 439. Tersedia pada: <https://doi.org/10.1007/s42979-023-01926-7>.

Hackers use SQL injection bug in BillQuick billing app to deploy ransomware (tanpa tanggal). Tersedia pada: <https://therecord.media/hackers-use-sql-injection-bug-in-billquick-billing-app-to-deploy-ransomware> (Diakses: 23 September 2025).

Hutchins, E.M., Cloppert, M.J. dan Amin, R.M. (2011) "Intelligence-Driven Computer Network Defense Informed by Analysis of Adversary Campaigns and Intrusion Kill Chains."

Ilascu, I. (tanpa tanggal) WooCommerce fixes vulnerability exposing 5 million sites to data theft, *BleepingComputer*. Tersedia pada: <https://www.bleepingcomputer.com/news/security/woocommerce-fixes-vulnerability-exposing-5-million-sites-to-data-theft/> (Diakses: 23 September 2025).

Jayalaxmi, P.L.S. dkk. (2022) "Machine and Deep Learning Solutions for Intrusion Detection and Prevention in IoTs: A Survey," *IEEE Access*, 10, hlm. 121173–121192. Tersedia pada: <https://doi.org/10.1109/ACCESS.2022.3220622>.

Kumar, A., Simha, J.B. dan Agarwal, R. (2024) "Machine Learning-Based Web Application Firewall for Real-Time Threat Detection," dalam *2024 IEEE Conference on Engineering Informatics (ICEI)*. 2024 *IEEE Conference on Engineering Informatics (ICEI)*, Melbourne, Australia: IEEE, hlm. 1–8. Tersedia pada: <https://doi.org/10.1109/ICEI64305.2024.10912239>.

Lundgren, B. dan Möller, N. (2019) "Defining Information Security," *Science and Engineering Ethics*, 25(2), hlm. 419–441. Tersedia pada: <https://doi.org/10.1007/s11948-017-9992-1>.

netfilter/iptables project homepage - The netfilter.org "libnetfilter_queue" project. Tersedia pada: https://www.netfilter.org/projects/libnetfilter_queue/index.html (Diakses: 25 September 2025).

Nwankwo, C.P. dkk. (2024) "Improving Open-Source Network Security Tooling for the Corporate Enterprise," dalam *2024 IEEE 5th International Conference on Electro-Computing Technologies for Humanity (NIGERCON)*. 2024 *IEEE 5th International Conference on Electro-Computing Technologies for Humanity (NIGERCON)*, Ado Ekiti, Nigeria: IEEE, hlm. 1–5. Tersedia pada: <https://doi.org/10.1109/NIGERCON62786.2024.10927387>.

OWASP Top Ten | OWASP Foundation (2023). Tersedia pada: <https://owasp.org/www-project-top-ten/> (Diakses: 23 September 2025).

PayloadsAllTheThings/SQL Injection/Intruder/SQL-Injection at master · swisskyrepo/PayloadsAllTheThings (2025) *GitHub*. Tersedia pada: <https://github.com/swisskyrepo/PayloadsAllTheThings/blob/master/SQL%20Injection/Intruder/SQL-Injection> (Diakses: 26 September 2025).

Prakash, A. dkk. (2023) "Model-Aware XGBoost Method Towards Optimum Performance of Flexible Distributed Raman Amplifier," *IEEE Photonics Journal*, 15(4), hlm. 1–10. Tersedia pada: <https://doi.org/10.1109/JPHOT.2023.3286272>.

Sharma, A. (tanpa tanggal) *Django fixes SQL Injection vulnerability in new releases*, *BleepingComputer*. Tersedia pada: <https://www.bleepingcomputer.com/news/security/django-fixes-sql-injection-vulnerability-in-new-releases/> (Diakses: 23 September 2025).

Tanang Anugrah, F., Ikhwan, S. dan Gusti A.G, J. (2022) "Implementasi Intrusion Prevention System (IPS) Menggunakan Suricata Untuk Serangan SQL Injection," *Techne : Jurnal Ilmiah Elektroteknika*, 21(2), hlm. 199–210. Tersedia pada: <https://doi.org/10.31358/techne.v21i2.320>.

Wiens, M. dkk. (2025) "A Tutorial and Use Case Example of the EXTREME Gradient Boosting (XGBOOST) Artificial Intelligence Algorithm for Drug Development Applications," *Clinical and Translational Science*, 18(3), hlm. e70172. Tersedia pada: <https://doi.org/10.1111/cts.70172>.

LAMPIRAN